Highly Reliable Anonymous Cloud Synchronization

Zhen Liu, Yi Zhao, *Member, IEEE*, Lei Kang, Zijian Zhang, *Senior Member, IEEE*, Yinggui Wang, and Liehuang Zhu, *Senior Member, IEEE*

***Abstract*—Password-based cloud storage (PBCS) allows users to outsource datasets to the cloud server and synchronize the corresponding data. However, there are two key challenges. *First*, they lack anonymity, making user activities linkable to public identities. Although integrating anonymous password-authenticated key exchange (APAKE) can achieve anonymity, it relies on private information retrieval (PIR), which causes high communication and computation overhead. Additionally, existing APAKE protocols are vulnerable to online password-guessing attacks (PGA). *Second*, PBCS cannot ensure reliable synchronization, where malicious/victimized servers may roll back user data undetectably. To address these issues, we propose HRACS, a highly reliable and anonymous cloud synchronization scheme, based on our newly designed PIR-free APAKE. Specifically, PIR-free APAKE replaces PIR with a lightweight pseudonym-signature mechanism, reducing communication overhead from linear to constant. The deterministic pseudonym also enables anonymous rate-limiting, mitigating online PGA. Moreover, HRACS simultaneously supports reliable synchronization of datasets by encrypting each item with authenticated encryption and computing a fingerprint of the entire dataset. Each item carries a binding proof to the fingerprint, allowing the user to detect any unauthorized modifications (e.g., reordering or rollback). Experimental results demonstrate that our PIR-free APAKE achieves competitive computation efficiency, reducing the overhead by up to 68.29% compared with existing APAKE protocols. Regarding communication, the bandwidth requirement remains constant and is reduced by up to 99.96% relative to PIR-based APAKE protocols when scaling to 20,000 users. Moreover, HRACS strengthens security without sacrificing performance, incurring overhead comparable to existing PBCS schemes.**

***Index Terms*—Anonymous authentication, cloud synchronization, password-based cloud storage (PBCS).**

I. Introduction

According to the projection provided by Fortune Business Insights [1], the cloud storage market size is expected to grow from USD 197.8 billion in 2026 to USD 809.99 billion by 2034. Although cloud storage is very convenient and popular, it presents serious security issues as users lose physical control over their data after outsourcing. To prevent unauthorized synchronization of the user data, most cloud service providers (e.g., Google Drive [2] and iCloud [3]) implement default encryption at rest to improve security. Specifically, data uploaded to the cloud is encrypted by the cloud server using a secret key [4], [5] before long-term storage. However, since the encryption is performed by the cloud server itself, it inherently gains access to the plaintext of user data. This poses a high risk when the cloud server actively or passively behaves maliciously, as it may extract, leak, or even modify the data without the user's consent.

To enhance user control over data, several studies [6], [7], [8], [9] propose user-side encryption. Specifically, using a locally managed secret key of authenticated encryption (AE) [10], users can encrypt data into a ciphertext for data confidentiality and generate a tag over the ciphertext for data integrity.[1] Among these studies, password-based cloud storage (PBCS) [4], [8] has emerged as a promising solution, where the secret key is deterministically derived from the user's low-entropy password. Users only need to remember a single password, eliminating the requirement for local key management. Beyond key derivation, the password is also used for user authentication (i.e., password-based authentication), facilitating data synchronization across different devices (e.g., smartphones and laptops) anytime and anywhere.

Despite the above advantages, existing PBCS schemes still have obvious defects in *anonymity* and *reliability*. First, due to the lack of anonymous authentication, adversaries can link users' synchronization activities to their public identities (e.g., usernames or e-mail addresses) [11]. This can lead to serious privacy violations. To address this issue, anonymous password-authenticated key exchange (APAKE) [12], [13], [14], [15], [16], [17], [18], [19] has been adopted as a solution. Through private information retrieval (PIR) [16], [20], APAKE can achieve anonymity by enabling password-based authentication without disclosing users' public identities. Upon successful authentication, the user and the server can exchange a session key for secure communication[2] (i.e., secure key exchange) throughout the session. Since PIR requires that the list containing password-related key exchange entries of all registered users must be transmitted to every user, the communication overhead increases linearly with the number of registered users. This imposes a significant scalability bottleneck in large-scale deployments. To address this issue,

Received 15 December 2025; revised 2 August 2026; accepted 20 August 2026. Date of publication 1 September 2026; date of current version 13 October 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62472036, in part by Beijing Nova Program and the Xiaomi Young Scholar, in part by the Open Research Fund of State Key Laboratory of Internet Architecture under Grant HLW2025MS04, and in part by NSFC under Grant U2468205. The associate editor coordinating the review of this article and approving it for publication was Dr. Shuoyao Wang. *(Corresponding author: Yi Zhao.)*

Zhen Liu, Yi Zhao, Lei Kang, Zijian Zhang, and Liehuang Zhu are with the School of Cyberspace Science and Technology, Beijing Institute of Technology, Beijing 100081, China (e-mail: zhenliu@bit.edu.cn; zhaoyi@bit.edu.cn; 3120256092@bit.edu.cn; zhangzijian@bit.edu.cn; liehuangz@bit.edu.cn).

Yinggui Wang resides in Beijing 100081, China (e-mail: wyinggui@gmail.com).

This article has supplementary downloadable material available at https://doi.org/10.1109/TIFS.2026.3729510, provided by the authors.

Digital Object Identifier 10.1109/TIFS.2026.3729510

[1]Data integrity means that any modification to the ciphertext causes tag verification failure.

[2]Here, secure communication refers to ensuring message confidentiality, integrity, and that the communicating party is the intended legitimate entity, rather than an impersonator.

1556-6021 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on October 22,2026 at 21:15:32 UTC from IEEE Xplore. Restrictions apply.

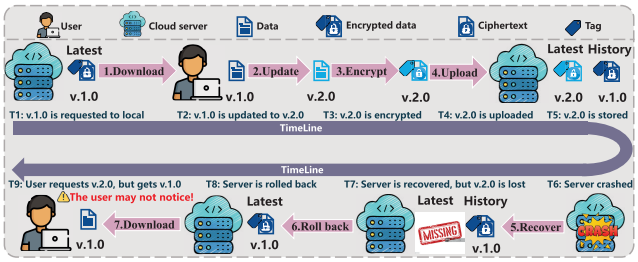


Fig. 1. Cloud synchronization failure caused by incidents (e.g., server crashes). The user initially downloads version 1.0 of a data item (e.g., a file) from the cloud server. After updating the data to version 2.0, the user encrypts and uploads it back to the server, which accepts it as the latest version and archives version 1.0 in the history cache. However, before version 2.0 is permanently stored, the server experiences a crash. Upon recovery, version 2.0 is lost, and the cloud server may silently revert to version 1.0, presenting it as the most recent version. The user may accept this outdated version without notice.

some alternatives have been proposed [17], [18], [19] to avoid PIR. However, they often rely on computationally expensive cryptographic primitives, such as zero-knowledge proofs (ZKP) [21] and homomorphic encryption (HE) [22]. Therefore, existing APAKE protocols fail to achieve low cost in terms of communication and computation. Worse still, existing APAKE protocols lack robustness since anonymity inherently conflicts with rate-limiting [23],[3] making them vulnerable to online password-guessing attacks (PGA) [24].

In addition to the challenges in anonymity, reliability is also difficult to guarantee. As illustrated in Fig. 1, the cloud server may roll back users' latest data to a previous version without notifying users. Notable incidents include GitLab database outage [25], Destiny 2 rollback event [26], and GitHub status disruption [27]. The situation becomes even worse when service providers choose not to disclose them, typically to protect their reputation or avoid liability. Even widely adopted platforms like Apple's iCloud [3] have experienced synchronization failures [28], [29], [30], without official acknowledgment, reinforcing the necessity for users to verify synchronization consistency independently. In fact, this requirement stems from a limitation of AE: both outdated and current data versions are encrypted with the same AE key, preventing users from distinguishing fresh data from stale data. A simple solution is to encrypt different data versions with distinct keys. However, this introduces a key management burden that grows linearly with the number of data versions [6].

To address the above issues, we propose HRACS, a highly reliable anonymous cloud synchronization scheme, which incorporates our newly designed PIR-free APAKE protocol as an underlying component. Unlike existing APAKE protocols that rely on PIR or computationally expensive primitives to achieve anonymity and secure key exchange, our PIR-free APAKE eliminates the requirement to implement PIR by using a novel and more lightweight pseudonym-signature binding mechanism, thus possessing the characteristics of strong anonymity, high robustness, and low cost. To simultaneously achieve both anonymity and reliability, our PIR-free APAKE employs an oblivious pseudorandom function (OPRF) [31][4] to convert the low-entropy password into a high-entropy string, dubbed the *entropy-enhanced seed*, in an interactive manner. After the interaction between the user and the server, this seed is further deterministically used to derive a pseudonym (jointly with the user's public identity), a digital signature key pair (i.e., public and private keys), and a data encryption key (DEK).

Regarding anonymity, the user sends the pseudonym and public key to the cloud server during the registration phase, so that subsequent authentication only requires the user to submit the pseudonym along with a single signed key exchange entry.[5] Then the server retrieves the corresponding public key to authenticate the user and complete the secure key exchange. Therefore, the server no longer needs to send a list of entries to each user, and HRACS achieves a communication overhead that is independent of the number of registered users. Since the OPRF key is protected by the TEE and the entropy-enhanced seed is of high-entropy, adversaries cannot link the pseudonym to the user's public identity, thereby achieving anonymity. Moreover, the deterministic pseudonym enables anonymous rate-limiting to mitigate online PGA, achieving robustness. Notably, HRACS attains all these advantages without relying on any computationally expensive cryptographic primitives, thereby achieving low cost.

In addition to anonymity, our HRACS can also guarantee reliability. Specifically, the user first utilizes the DEK generated from the entropy-enhanced seed to encrypt each data item and generate a ciphertext-tag pair to ensure confidentiality and integrity. Subsequently, the user organizes all tags into a tag-vector in a preconfigured order, generates a unique fingerprint for the vector through the commitment algorithm as in [33], and maintains the fingerprint (e.g., storing it in the anonymous blockchain [34]). After the user uploads all ciphertexts and tags, the server uses the opening (i.e., proof generation) algorithm, generating a proof for each tag to reduce the user's computation overhead. When the user downloads a data item, the corresponding tag and proof will be downloaded synchronously. If the item is not the latest (e.g., the server rolls back or the server is controlled by hackers to maliciously tamper with the data), the user can discover reliability failures through proof errors (e.g., location-based proof and fingerprint consistency checks provided by [33]) or tag inconsistency (e.g., integrity verification supported by AE).

Our contributions are summarized as follows:

- To the best of our knowledge, our proposed HRACS is the first PBCS framework that simultaneously achieves user anonymity and reliable cloud synchronization.
- We design a PIR-free APAKE protocol, achieving low cost (i.e., communication overhead independent of the number of registered users and free from computationally expensive cryptographic primitives) and robustness (i.e., mitigating online PGA).

[3]Rate-limiting [23] is a mechanism to prevent online PGA, which relies on public identity to restrict the number of login attempts.

[4]The OPRF key is securely stored inside a trusted execution environment (TEE) [32] on the server, rendering it inaccessible even to the server itself.

[5]Specifically, the user signs the key exchange entry using the private key, and the server verifies the signature using the corresponding public key.

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on October 22,2026 at 21:15:32 UTC from IEEE Xplore. Restrictions apply.

- Via our PIR-free APAKE, a commitment-driven reliable cloud synchronization mechanism is proposed simultaneously, which constructs a tag-vector and generates a fingerprint over the vector, exposing inconsistent synchronization through proof errors or tag verification failure.
- We formally define and prove the security of HRACS under the random oracle model (ROM).
- Our experimental results demonstrate that the proposed PIR-free APAKE achieves competitive computation efficiency, reducing the computation overhead by up to 68.29% compared with existing APAKE protocols. Moreover, its communication overhead remains constant as the number of registered users increases, achieving a reduction of up to 99.96% over PIR-based APAKE approaches at a scale of 20,000 users. Meanwhile, HRACS incurs computation and communication overhead comparable to that of state-of-the-art PBCS schemes, while additionally providing strong anonymity and reliable synchronization guarantees. Furthermore, the anonymous blockchain evaluation demonstrates the economic viability and sustainability of HRACS in large-scale user settings.

The remainder of the paper is organized as follows: Section II reviews the related literature. Section III introduces the necessary preliminaries and presents the problem statement. Section IV describes our proposed HRACS. The security definition and proof for HRACS are provided in Section V. The performance evaluation of HRACS is presented in Section VI, and Section VII concludes the paper.

II. Related Work

In this section, we review existing schemes on APAKE and secure cloud storage with regard to anonymity and reliability, analyze their security and efficiency properties, and identify the gaps that motivate our study. Furthermore, we compare these schemes with our study to highlight our innovations.

A. Anonymous Password-Authenticated Key Exchange

The APAKE protocols [12], [13], [14], [15], [16], [17], [18], [19] enable a user and a server to securely exchange a session key using only a password, while preserving both user anonymity and password security. Viet et al. [12] introduced the first APAKE by combining a secure password-authenticated key exchange (PAKE) [20] with oblivious transfer [35] (i.e., a primitive enabling PIR for anonymity). Shin et al. [13] further reduced the computation overhead of the protocol in [12] by 50% through cutting its number of modular exponentiations. Regrettably, protocols in [12] and [13] are vulnerable to offline PGA, as the adversary can impersonate the server to obtain password-related information from the user, allowing offline verification of password guesses. To resist the impersonation attack, Yang and Zhang [14] enable the server to compute a valid authentication proof using the password shared with the user during registration. However, apart from the password, their solution requires users to additionally memorize a secret index used for PIR, which is impractical. To overcome this limitation, Vasco et al. [16] proposed deterministically deriving the index from the password using OPRF [31], or encrypting the index using identity-based encryption (IBE) [36] with the password as the secret key, allowing secure recovery through decryption. Despite these advances, all of these protocols [12], [13], [14], [16] incur communication overhead linear in the number of registered users, due to their reliance on PIR.

TABLE I

Comparison of Existing APAKE Protocols.[†]

Scheme	Comm.	Comput.	Online PGA	Offline PGA
[12]	△	▲	△	△
[13]	△	▲	△	△
[14]	△	▲	△	▲
[15]	△	▲	△	▲
[16]-OPRF	△	△	△	▲
[16]-IBE	△	△	△	▲
[17]	▲	△	△	▲
[18]	▲	△	△	▲
[19]	▲	△	△	▲
Our work	▲	▲	▲	▲

[†] **Comm.**: communication cost is independent of the registered user number, **Comput.**: the protocol does not rely on computationally expensive cryptographic primitives (e.g., HE or ZKP), **Online/Offline PGA**: the scheme is secure against online/offline PGA. Satisfied and unsatisfied properties are denoted by ▲ and △, respectively.

To achieve constant communication overhead, Yang et al. [17] proposed the first APAKE protocol without PIR, which combines PAKE with an anonymous credential [37] that enables user authentication without exposing public identities. However, their protocol relies on computationally expensive cryptographic primitives, including ZKP [21] and HE [22]. To improve efficiency, Zhang et al. [19] replaced HE with a more lightweight primitive, algebraic message authentication code (MAC) [38], but ZKP remained. Moreover, the protocols [12], [13], [14], [15], [16], [17], [19] are all vulnerable to online PGA, due to the absence of effective mechanisms for rate-limiting in anonymous settings. The only work relevant to this issue [18] requires users' timely assistance, which is impractical.

To overcome the above limitations, we propose a novel PIR-free APAKE that simultaneously achieves constant communication overhead, avoids employing computationally expensive primitives, and resists both online and offline PGA. This is made possible by using a novel pseudonym–signature binding mechanism. In detail, Table I summarizes the comparison between our work and existing APAKE protocols.

Another line of related research achieves user anonymity through verifiable random function (VRF) [39], [40], [41]. These approaches are primarily tailored for blockchain systems, where public keys serve as long-term identities, and ephemeral identifiers are derived via VRF to ensure unlinkability across different sessions or activities. However, VRF verification inherently requires revealing the underlying input. This constraint renders such schemes unsuitable for password-based settings, where the input (i.e., the user's password) must remain confidential and cannot be disclosed to any party. In contrast, our PIR-free APAKE framework is built on OPRF, which eliminates this vulnerability by blinding the password prior to interaction. This critical distinction makes our approach uniquely suited for password-authenticated scenarios.

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on October 22,2026 at 21:15:32 UTC from IEEE Xplore. Restrictions apply.

TABLE II
FEATURE COMPARISON OF EXISTING SECURE CLOUD SYNCHRONIZATION SCHEMES.[†]

Scheme	PBCS	AA	BQ	DU	RCS	SS
SiRiUS [42]	○	○	○	○	●	●
DepSky [43]	○	○	○	○	○	○
Mylar [44]	○	○	○	○	○	●
Burnbox [56]	○	○	○	○	○	●
PHE [4]	●	○	○	○	○	●
OKMS [6]	○	○	○	●	○	●
ETS [57]	○	○	○	○	○	●
Metal [46]	○	●	○	○	○	○
PFS [45]	○	○	○	○	○	●
Titanium [47]	○	●	●	○	●	○
DPaSE [48]	●	○	○	○	○	○
PLCS [58]	●	○	○	○	○	●
WBP [49]	●	○	○	○	○	●
FPBE [50]	●	○	○	○	○	●
CSS [8]	●	○	○	●	○	●
USS [7]	○	○	●	●	●	●
Li et al. [59]	●	○	●	●	●	●
Our work	●	●	●	●	●	●

[†]**PBCS**: password-based cloud storage, **AA**: anonymous authentication, **BQ**: batch queries, **DU**: data update, **RCS**: reliable cloud synchronization, **SS**: single server deployment. Missing and achieved features are denoted by ○and ●, respectively.

TABLE III
NOTATIONS

Notation	Explanation
λ	The security parameter
$\lvert\mathcal{PW}\rvert$	The cardinality of the user password space
$[1, n]$	A set $\{1, \cdots, n\}$
$\mathsf{negl}(\lambda)$	The negligible function
$\mathbb{G}, \mathbb{G}_T$	Two multiplicative cyclic groups
g	The generator of $\mathbb{G}$
p	The prime order of $\mathbb{G}$ and $\mathbb{G}_T$
e	A bilinear pairing: $\mathbb{G} \times \mathbb{G} \rightarrow \mathbb{G}_T$
$x\|\|y$	The concatenation of two bit-strings x and y
$x \leftarrow a$	Assign a to a variable x
$x \leftarrow\!\!\$\ S$	Pick an element x from a set S uniformly at random
(pk, sk)	The signature key pair
ρ	The pseudonym of the user
k_o	The secret key of OPRF
k_{se}	The session key
k_{ae}	The AE key
$\lvert l\rvert$	The bit-string length of the parameter l
$\mathcal{H}$	A hash function: $\{0,1\}^* \rightarrow \mathbb{G}$
H	A hash function: $\{0,1\}^* \rightarrow \{0,1\}^{256}$
H_1	A hash function: $\{0,1\}^* \rightarrow \{0,1\}^{\lvert\rho\rvert}$
H_2	A hash function: $\{0,1\}^* \rightarrow \{0,1\}^{\lvert sk\rvert}$
H_3	A hash function: $\{0,1\}^* \rightarrow \{0,1\}^{\lvert k_{se}\rvert}$
H_4	A hash function: $\{0,1\}^* \rightarrow \{0,1\}^{\lvert k_{ae}\rvert}$

B. Secure Cloud Synchronization

Secure cloud synchronization enables users to upload their datasets to a remote cloud server for long-term storage and later download them when needed, all while preserving data confidentiality and integrity through cryptographic techniques. Existing secure cloud synchronization schemes can be categorized into key-management-based cloud storage (KMBCS) [6], [7], [42], [43], [44], [45], [46], [47] and password-based cloud storage (PBCS) [4], [8], [48], [49], [50].

In KMBCS, the user manages the encryption key locally. Among such schemes, SiRiUS [42] constructs a hash tree [51] from all encrypted data items for reliability, allowing users to detect synchronization inconsistencies by comparing the original root hash with the one rebuilt from the retrieved data items. Yet, SiRiUS does not consider anonymity. To resolve this limitation, Titanium [47] employs Tor [52] (i.e., a network enabling anonymous communication) to conceal the user's public identity. Moreover, Titanium achieves reliability by distributing the AE tag of each data item across multiple servers using Shamir's secret sharing [53], assuming that at least one server is honest. However, KMBCS suffers from poor portability, as users must securely store and carry their encryption keys wherever they synchronize the data. In contrast, PBCS eliminates this problem by integrating password-based authentication [54] and key derivation [55], enabling users to synchronize the data anytime and anywhere while preventing identity impersonation. However, none of the existing PBCS schemes can ensure both anonymity and reliability.

Recently, several version-controlled secure cloud storage schemes have been proposed to provide confidentiality, integrity, and reliability. Chen et al. proposed USS [7], which combines updatable encryption (UE) [60], Pedersen commitments [61], and commitment-based fingerprints to achieve these properties. However, USS relies heavily on costly asymmetric operations, such as bilinear pairings, and lacks standardized library support, limiting its practical deployment. Li et al. [59] proposed a fine-grained version-control scheme based on a Merkle DAG [62]. It records encrypted file modifications in the DAG and signs the root to verify integrity and version freshness. However, users must maintain the complete DAG and locally apply all accumulated modifications during retrieval, resulting in non-constant user-side overhead.

To overcome the above limitations, we propose HRACS, a novel PBCS framework that ensures both anonymity and reliability. Beyond these two properties, we also survey additional desirable features. As illustrated in Table II, HRACS also supports data updates, efficient batch queries (i.e., leveraging an aggregate proof to verify the reliability of multiple data items in a single request), and single-server deployment (i.e., no multi-server collaboration, avoiding heavy communication cost). Moreover, compared with studies [7], [59], HRACS replaces the costly UE–Pedersen construction with AE, enabling lightweight implementation using standard cryptographic libraries. By decoupling state freshness from data structures, it also achieves constant user-side update overhead through dual verification. Unlike works addressing anonymity or version control separately, HRACS integrates PIR-free APAKE with lightweight version control, making it the first secure password-based cloud storage framework to provide both anonymity and reliability.

III. PRELIMINARY

In this section, we first present the basic notations and theory used to build HRACS. Then, we introduce the system model, threat model, and design goals of HRACS.

A. Notations

The notations used in this paper are illustrated in Table III. Additionally, for an algorithm A, $(y_1, y_2, \cdots) \leftarrow A(x_1, x_2, \cdots)$

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on October 22,2026 at 21:15:32 UTC from IEEE Xplore. Restrictions apply.

or $A(x_1, x_2, \cdots) \rightarrow (y_1, y_2, \cdots)$ indicates that A takes $(x_1, x_2, \cdots)$ as input and outputs $(y_1, y_2, \cdots)$. By $\mathsf{Exp}^{\mathsf{Property}}_{\Pi,\mathcal{A}}$, we denote the security experiment of a property $\mathsf{Property}$, parameterized by a scheme Π and a probabilistic polynomial-time (PPT) adversary $\mathcal{A}$. Furthermore, we let $(y_1, y_2, \cdots) \leftarrow \mathcal{A}^{O_1, O_2, \cdots}(x_1, x_2, \cdots)$ denote $\mathcal{A}$ on inputs $(x_1, x_2, \cdots)$, with oracle access to $O_1, O_2, \cdots$, and generating outputs $(y_1, y_2, \cdots)$. Finally, we use $\mathsf{Adv}^{\mathsf{Property}}_{\Pi}(\mathcal{B})$ to denote the advantage (i.e., probability) of $\mathcal{A}$ that runs a PPT algorithm $\mathcal{B}$ to break the $\mathsf{Property}$ of scheme Π.

B. Basic Theory

1) Bilinear Pairings: Given $(\mathbb{G}, \mathbb{G}_T, g, p)$, a bilinear pairing $e : \mathbb{G} \times \mathbb{G} \rightarrow \mathbb{G}_T$ satisfies the properties below:

- Bilinearity: $\forall g \in \mathbb{G}$, $(a, b) \in \mathbb{Z}_p$, $e(g^a, g^b) = e(g, g)^{ab}$.
- Non-degeneracy: $\forall g \in \mathbb{G}$, $e(g, g) \neq 1$.
- Computability: It is efficient to compute e.

2) Decisional Diffie-Hellman (DDH) Assumption: Given $(\mathbb{G}, g, p, g^x, g^y)$, where $x \leftarrow_{\$} \mathbb{Z}_p$, $y \leftarrow_{\$} \mathbb{Z}_p$, it is computationally hard to distinguish g^{xy} from $z \leftarrow_{\$} \mathbb{G}$.

3) Oblivious Pseudorandom Functions (OPRF) [31]: An OPRF is a two-party protocol between a sender $\mathcal{S}$, who holds a key k_o, and a receiver $\mathcal{R}$, who provides an input x. It enables $\mathcal{S}$ to evaluate a pseudorandom function $\mathsf{F}_{k_o}(x)$ using k_o, but learning nothing about x, and $\mathcal{R}$ receives only the output $\mathsf{F}_{k_o}(x)$, without gaining any information about k_o.

4) Digital Signature (DS): A DS scheme can ensure the authenticity, integrity, and non-repudiation of data. Formally, a DS scheme consists of the following three algorithms:

- $\mathsf{KeyGen}(\lambda) \rightarrow (pk, sk)$. Given the security parameter λ, it outputs a public-private key pair (pk, sk).
- $\mathsf{Sign}(sk, m) \rightarrow \sigma$. Given a secret key sk and a message m, it outputs a signature σ for m.
- $\mathsf{Vrfy}(pk, m, \sigma) \rightarrow 0/1$. Given a public key pk, a message m and a signature σ, it outputs 1 if σ is valid (i.e., σ is the signature of m), and 0 otherwise.

5) Authenticated Encryption (AE) [10]: AE is a cryptographic primitive that provides both data confidentiality and integrity. Formally, an AE scheme consists of the following three algorithms:

- $\mathsf{KeyGen}(\lambda) \rightarrow k_{ae}$. Given the security parameter λ, it outputs a secret key k_{ae}.
- $\mathsf{Enc}(k_{ae}, m) \rightarrow (c_{ae}, \tau)$. Given a secret key k_{ae} and a message m, it outputs a ciphertext c_{ae} and a tag τ.
- $\mathsf{Dec}(k_{ae}, c_{ae}, \tau) \rightarrow m/abort$. Given a secret key k_{ae}, a ciphertext c_{ae} and a tag τ, it outputs m if τ is valid (i.e., τ is the tag of m), and aborts otherwise.

6) Vector Commitment (VC) [33]: A VC scheme allows a committer to generate a commitment for a vector as a fingerprint, enabling the verification of individual items at specific indexes without revealing the entire vector.

A homomorphic VC (HVC) [33] operates as follows:

- $\mathsf{Setup}(\lambda, n) \rightarrow pp$. Given the security parameter λ and length n of the vector, it chooses $(\mathbb{G}, \mathbb{G}_T, g, p, e)$, and then picks $z_1, \cdots, z_n \leftarrow_{\$} \mathbb{Z}_p$. For $i \in [1, n]$, it sets $h_i \leftarrow g^{z_i}$. For $i, j \in [1, n], i \neq j$, it sets $h_{i,j} \leftarrow g^{z_i z_j}$. Finally, it outputs the public parameter

$$pp \leftarrow (\mathbb{G}, \mathbb{G}_T, e, g, p, \{h_i\}_{i \in [1,n]}, \{h_{i,j}\}_{i,j \in [1,n], i \neq j}).$$

- $\mathsf{Com}(\boldsymbol{m}) \rightarrow C$. Given a vector $\boldsymbol{m}$ of length n, it outputs a fingerprint $C \leftarrow h_1^{m_1} \cdot h_2^{m_2} \cdots h_n^{m_n}$.
- $\mathsf{Open}(i, m_i) \rightarrow \pi_i$. Given a position i and an element m_i, it outputs a proof

$$\pi_i \leftarrow \prod_{j=1, j \neq i}^{n} h_{i,j}^{m_j} = \left(\prod_{j=1, j \neq i}^{n} h_j^{m_j} \right)^{z_i}.$$

- $\mathsf{Vrfy}(C, m_i, i, \pi_i) \rightarrow 0/1$. Given a fingerprint C, an element m_i, a position i, and a proof π_i, it outputs 1 if $e(C/h^{m_i}, h_i) = e(\pi_i, g)$ holds, and 0 otherwise.
- $\mathsf{ComHom}(C, C') \rightarrow C''$. Given two fingerprints C and C', it outputs a new fingerprint $C'' \leftarrow C \cdot C'$.
- $\mathsf{OpenHom}(\pi, \pi') \rightarrow \pi''$. Given two proofs π and π', it outputs a new proof $\pi'' \leftarrow \pi \cdot \pi'$.

7) Anonymous Blockchain [34]: Blockchain [63] is a decentralized ledger that ensures tamper-evident record storage. To protect user privacy, anonymous blockchain systems incorporate cryptographic techniques to unlink users' public identities from their stored records.

8) Password-Based Cloud Storage (PBCS): PBCS enables cloud synchronization by deterministically deriving a secret key from the user's password. As illustrated in Fig. 2, the key is used to encrypt data into a ciphertext and generate an authentication tag over the ciphertext. The ciphertext–tag pair is uploaded to the cloud for storage. Upon downloading, the user recomputes the tag using the same key and compares it against the received one. If they match, the ciphertext is decrypted to recover the original data; otherwise, the process is aborted due to the integrity verification failure.

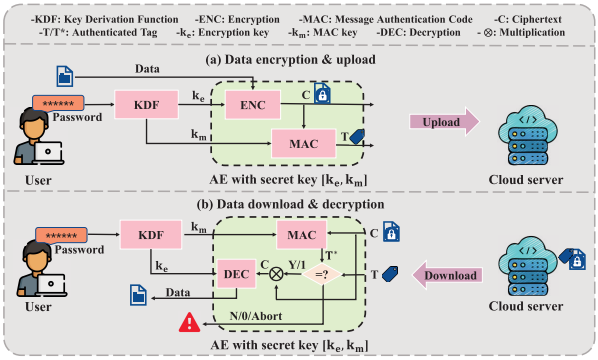


Fig. 2. Overview of PBCS.

9) Trusted Execution Environment (TEE) [32]: TEE technology (e.g., Intel SGX [64]) provides hardware-enforced isolation for sensitive computations. It creates an enclave that protects code and data in the enclave page cache (EPC), preventing access by privileged system software, including the operating system, hypervisor, drivers, and BIOS. Enclave memory is further protected by a hardware-based memory encryption engine to preserve confidentiality and integrity against software compromise. For persistent storage, SGX provides a sealing mechanism that encrypts enclave data using a key derived from the enclave measurement (e.g., its structural

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on October 22,2026 at 21:15:32 UTC from IEEE Xplore. Restrictions apply.

code hash) or identity, ensuring that only an authorized enclave can recover the sealed data.

C. System Model

Our proposed HRACS scheme involves two parties:

- **User $\mathcal{U}$.** The user operates client devices (e.g., smartphones, tablets, or laptops) to interact with the cloud server. As the original dataset owner, the user can anonymously log into the cloud server using only a password. To reduce the local storage burden, the user uploads the datasets to the cloud server. After uploading, the user can synchronize data anytime and anywhere via password-based authentication. In our HRACS, the user side does not deploy public key infrastructure (PKI).[6]
- **Cloud server $\mathcal{CS}$.** Operated by the service provider, the cloud server is equipped with ample storage and computation resources. It is responsible for storing and maintaining the user's outsourced datasets. The cloud server is equipped with a TEE, ensuring that code and data inside isolated hardware remain confidential and tamper-resistant, even if the server itself is compromised [32], [64].[7] Additionally, the cloud server is equipped with a PKI, i.e., the server holds a digital certificate issued by a trusted certificate authority, enabling users to verify and trust the server's public key.

D. Threat Model

We adopt a threat model, under which state-of-the-art PBCS schemes fail to preserve anonymity or reliability, whereas our HRACS remains secure. Specifically, we make the following assumptions to characterize this model:

- **Communication assumption.** User registration is protected by TLS secure channels, while user login occurs over a public channel, i.e., adversaries can eavesdrop, intercept, and redirect messages transmitted between the user and the cloud server.[8]
- **Adversary assumption.** Following [67], we assume a PPT adversary $\mathcal{A}$ with abilities as follows:
  – $\mathcal{A}$ can enumerate all items (i.e., candidate passwords) in the user password space (i.e., $|\mathcal{PW}|$).[9]
  – $\mathcal{A}$ can register as a legitimate user.
  – $\mathcal{A}$ can compromise the cloud server's storage (i.e., the datasets uploaded by the user and the registration table that records every user's pseudonym).
- **TEE assumption.** We assume that the underlying TEE guarantees the confidentiality and integrity of the code and data stored and executed inside the enclave, even if the cloud server is compromised. Notably, attacks that compromise the TEE (e.g., side-channel attacks) are beyond the scope of this work.

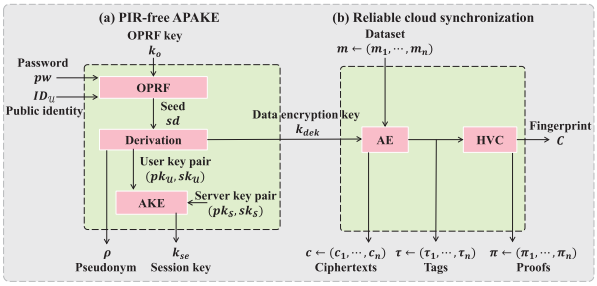


Fig. 3. Construction of HRACS.

E. Design Goals

Our HRACS is designed to achieve the goals below:

- **Authenticated key exchange (AKE) security**. The session key exchanged between the user and the cloud server should be indistinguishable from a random string.
- **Portability**. Only legitimately registered users can authenticate with the cloud server using a password and perform cloud synchronization anytime and anywhere.
- **Anonymity.** The adversary cannot link a user's real-world identity to its pseudonym. Notably, we do not consider unlinkability across different sessions or activities under a deterministic pseudonym, as the latter is intentionally introduced to support anonymity and server-side rate-limiting (i.e., resistance to online PGA), which is typically ignored by traditional APAKE protocols [12], [13], [14], [15], [16], [17].
- **Confidentiality**. Datasets uploaded to the cloud server should remain private from unauthorized parties.
- **Reliability**. Datasets uploaded by the user to the cloud server should remain intact, and any inconsistent synchronization must be detectable.
- **Robustness**. The adversary cannot successfully launch an online PGA under rate-limiting.
- **Low cost**. The design of HRACS should avoid introducing heavy computation and communication overhead.

IV. Our Proposed HRACS

In this section, we first present the construction of our HRACS framework, consisting of a novel PIR-free APAKE protocol and a reliable cloud synchronization mechanism. We then describe each component in detail.

A. Construction of HRACS

As illustrated in Fig. 3, our HRACS framework simultaneously achieves both anonymity and reliability by integrating a *PIR-free APAKE protocol* and a *reliable cloud synchronization mechanism*. First, to securely authenticate users with low-entropy passwords without exposing them to the cloud, we design a PIR-free APAKE protocol. Specifically, a user inputs a low-entropy password pw into an OPRF under the cloud server's key k_o, along with a public identity $ID_{\mathcal{U}}$. This process transforms the weak password pw into an entropy-enhanced

[6]PKI is a system that issues digital certificates for entities to establish trust in their public keys. The assumption of not deploying PKI on the user side eliminates the burden of maintaining digital certificates for users, making the system lightweight and practical, especially in resource-constrained environments, e.g., the Internet of Things.

[7]This assumption is practical, as major cloud providers already deploy TEE (e.g., Intel SGX on Azure [65] and AMD SEV on Google Cloud [66]).

[8]During the registration phase, the cloud server employs PKI to establish secure channels with users via the TLS protocol. In contrast, during the login phase of our HRACS framework, users authenticate and establish a session key with the cloud server for secure communication over the public channel via our novel PIR-free APAKE protocol.

[9]Since low-entropy passwords have small search spaces, adversaries can feasibly enumerate all candidate passwords.

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on October 22,2026 at 21:15:32 UTC from IEEE Xplore. Restrictions apply.

seed sd, enabling users to authenticate with short, memorable passwords while ensuring strong cryptographic security. The seed sd is used to derive a pseudonym ρ and a signature key pair $(pk_{\mathcal{U}}, sk_{\mathcal{U}})$ for the user. The pseudonym ρ conceals the user's public identity $ID_{\mathcal{U}}$, thereby achieving anonymity. The derived signature key pair $(pk_{\mathcal{U}}, sk_{\mathcal{U}})$ participates in an AKE with the server's signature key pair $(pk_{\mathcal{S}}, sk_{\mathcal{S}})$ to establish a session key k_{se} for secure communication. Further details are provided in Section IV-B. Second, to address the challenge of ensuring reliable dataset synchronization with an untrusted cloud server, we design a reliable cloud synchronization mechanism. Specifically, the seed generated in the PIR-free APAKE phase is further used to derive a data encryption key k_{dek} (i.e., DEK) for protecting the dataset $\boldsymbol{m} \leftarrow (m_1, \cdots, m_n)$. Each data item m_i $(i \in [1, n])$ is encrypted into a ciphertext c_i with an authenticated tag τ_i using the DEK k_{dek}, ensuring both confidentiality and integrity. Moreover, for each tag τ_i, the HVC generates a proof π_i and computes a fingerprint C over the tag vector $\boldsymbol{\tau} \leftarrow (\tau_1, \cdots, \tau_n)$. Together, the proofs $\boldsymbol{\pi} \leftarrow (\pi_1, \cdots, \pi_n)$ and fingerprint C enable reliability verification. More details are presented in Section IV-C.

Notably, in HRACS, several security-critical values, including the pseudonym ρ, signature key pair $(pk_{\mathcal{U}}, sk_{\mathcal{U}})$, and data encryption key k_{dek}, are derived from the same entropy-enhanced seed sd. This design may raise concerns that the compromise of one derived key could affect the security of other keys. To mitigate such cascading risks, HRACS enforces protection across three layers. First, we ensure cryptographic key separation. Although derived from the same seed sd, these keys are generated using distinct cryptographically secure one-way hash functions, making it computationally infeasible to correlate or deduce one key from another. Second, we employ hardware-level isolation by executing the OPRF under the cloud server's OPRF key k_o within a TEE. Since this key is strictly protected by the TEE, the 256-bit seed sd cannot be reconstructed externally. Moreover, exhaustive enumeration of sd is computationally infeasible due to its negligible success probability, i.e., 2^{-256}. Finally, we adopt an ephemeral key lifecycle policy, where all derived keys are generated strictly on demand and immediately erased upon operation completion, preventing persistent-storage leakage.

B. PIR-Free APAKE

Our PIR-free APAKE consists of three algorithms: Setup, Registration, and Login.

In Setup, the system generates parameters as follows:

Algorithm: Setup

Input: Security parameter λ[a]; signature scheme DS.

Step 1: Select a cyclic group $\mathbb{G}$ of prime order p with generator g.
Step 2: Sample an OPRF key $k_o \leftarrow\$ \mathbb{Z}_p$.
Step 3: Run KeyGen of DS to obtain the server's signature key pair $(pk_{\mathcal{S}}, sk_{\mathcal{S}})$.
Step 4: Set the public parameters as:

$$pp \leftarrow (\mathbb{G}, p, g, pk_{\mathcal{S}}, \mathcal{H}, \mathsf{H}, H_1, H_2, H_3).$$

Output: Public parameters pp; secret keys $(k_o, sk_{\mathcal{S}})$ stored in the TEE deployed on $\mathcal{CS}$.

[a]In cryptography, the security parameter controls the computational hardness of the scheme; 128-bit security informally means that the best known generic attack requires about 2^{128} basic operations.

TABLE IV
PARAMETERS INVOLVED IN PSEUDONYM GENERATION

Symbol	Description	Role
pw	User-selected password	Authentication secret
$ID_{\mathcal{U}}$	Public user identity	User identification
k_o	Server OPRF key	Password hardening
sd	Entropy-enhanced seed output by OPRF	Root secret

As illustrated in Fig. 4a, during Registration, a key challenge is how to securely handle the user's low-entropy password pw without exposing it to the cloud server. To address this challenge, the user and the server interactively execute an OPRF under the server's OPRF key k_o, transforming the password pw and public identity $ID_{\mathcal{U}}$ into an entropy-enhanced seed sd (i.e., a 256-bit high-entropy string). To prevent the leakage of the OPRF key k_o, it is stored inside a TEE, rendering it inaccessible even to the server itself. In this way, no adversary can recover the seed sd, ensuring that users can securely use passwords for authentication anytime and anywhere across different devices. Based on this design, a new challenge is to achieve anonymity and prevent impersonation attacks. To this end, the user deterministically derives a pseudonym ρ from the seed sd and public identity $ID_{\mathcal{U}}$. The pseudonym ρ is then stored on the anonymous blockchain. Table IV illustrates the parameters involved in pseudonym generation, facilitating quick reference. Security of the seed sd guarantees that the pseudonym ρ is unlinkable to the user's public identity $ID_{\mathcal{U}}$, thereby ensuring anonymity. Moreover, the determinism of the pseudonym ρ enables the server to enforce anonymous rate-limiting, thereby mitigating online PGA and achieving robustness. To further bind anonymity with user authentication, the user derives a signature key pair $(sk_{\mathcal{U}}, pk_{\mathcal{U}})$ from the seed sd and submits the pseudonym–public key pair $(\rho, pk_{\mathcal{U}})$ to the server, which then signs this pair with its private key $sk_{\mathcal{S}}$, generating a signature σ_ρ. The signature σ_ρ establishes a pseudonym–signature binding mechanism, enabling the server to verify that the presented pseudonym ρ corresponds to the public key $pk_{\mathcal{U}}$ during login. If the binding holds, the server can further verify that the user possesses the corresponding private key $sk_{\mathcal{U}}$, which can only be generated by the legitimate user, thereby preventing impersonation attacks.

To initiate Login, as illustrated in Fig. 4b, the user first retrieves the pseudonym ρ from the anonymous blockchain and sends it to the cloud server, allowing the server to identify the corresponding registration entry without learning the user's real identity $ID_{\mathcal{U}}$. To mitigate online PGA, the server enforces rate-limiting by initializing a counter cnt_ρ for the pseudonym ρ. Each login attempt increases the counter cnt_ρ, and if its value exceeds a predefined threshold cnt, the user's account is temporarily blocked. To ensure the authenticity of the key exchange entry, the server selects a random value $y \in \mathbb{Z}_p$, uses its private key $sk_{\mathcal{S}}$ to generate a signature $\sigma_{\mathcal{S}}$ over the random group element $Y \leftarrow g^y$, and returns the element-signature pair $(Y, \sigma_{\mathcal{S}})$ as the password exchange entry to the user. By verifying the signature $\sigma_{\mathcal{S}}$ using the server's public key $pk_{\mathcal{S}}$, the user is assured that the entry $(Y, \sigma_{\mathcal{S}})$ indeed originates from the server. The user then recovers the

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on October 22,2026 at 21:15:32 UTC from IEEE Xplore. Restrictions apply.

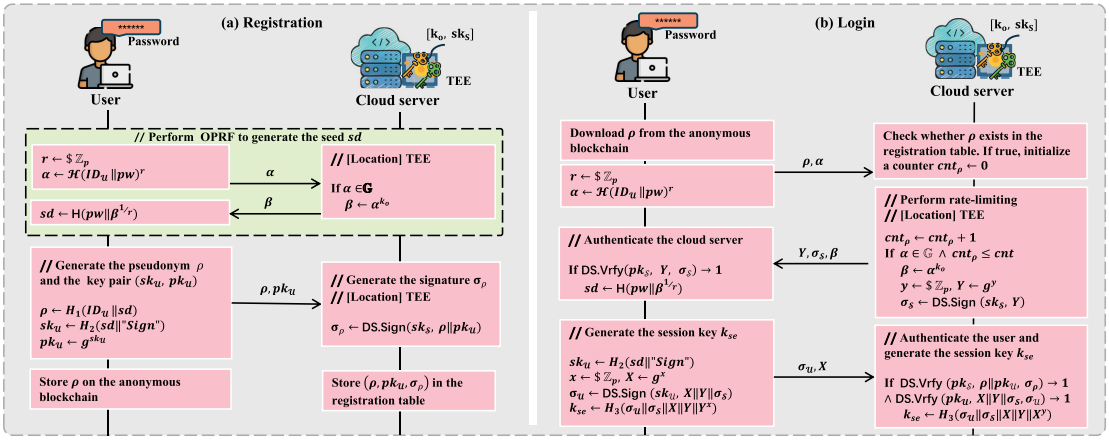


Fig. 4. PIR-free APAKE.

seed sd (by interactively performing OPRF with the server) to derive the secret key $sk_{\mathcal{U}}$, randomly samples $x \in \mathbb{Z}_p$, produces a signature $\sigma_{\mathcal{U}}$ over the random group element $X \leftarrow g^x$, and sends the element-signature pair $(X, \sigma_{\mathcal{U}})$ to the server as the password-related key exchange entry. The server retrieves the corresponding public key $pk_{\mathcal{U}}$ to verify both the pseudonym signature σ_ρ and the user's signature $\sigma_{\mathcal{U}}$, which jointly guarantee that the entry $(X, \sigma_{\mathcal{U}})$ originates from the intended user. If all the verifications succeed, both the user and the server can derive the shared session key k_{se} from the entries, enabling secure communication over public channels. During this process, the randomness of the group elements X and Y guarantees the freshness of the session and prevents replay attacks. Notably, since the server does not need to send a list of password exchange entries to the user for anonymity and key exchange (i.e., it does not rely on PIR), the communication overhead during login remains independent of the number of registered users. Moreover, our PIR-free APAKE protocol achieves these security properties without relying on computationally expensive primitives such as ZKP or HE. Therefore, it achieves low cost in terms of both communication and computation.

Additionally, we explicitly note that our PIR-free APAKE does not aim to prevent behavioral linkage across different sessions or activities under the same pseudonym. Such linkage is a direct consequence of adopting deterministic pseudonyms, which are necessary for server-side rate-limiting (i.e., online PGA mitigation). We argue that preserving the user's account security against brute-force takeover carries a higher priority than fully concealing cross-session/activity correlations from the server. Therefore, our PIR-free APAKE reflects a trade-off between strong unlinkability and account security.

C. Reliable Cloud Synchronization

Our reliable cloud synchronization mechanism consists of five algorithms: Setup, Upload, SingleQuery, BatchQuery, and Update.

In Setup, the system generates and publishes the public parameter $pp_{hvc} \leftarrow \mathsf{HVC.Setup}(\lambda, n)$.

As illustrated in Fig. 5a, to Upload a dataset $\boldsymbol{m} \leftarrow (m_1, \cdots, m_n)$ to the cloud server, the user first encrypts each data item m_i ($i \in [1, n]$) using DEK k_{dek}, which is derived from the seed sd generated in the PIR-free APAKE protocol. This process produces a ciphertext c_i and an authenticated tag τ_i for each data item m_i, thereby ensuring both data confidentiality and integrity. Subsequently, all tags are compacted into a vector $\boldsymbol{\tau} \leftarrow (\tau_1, \cdots, \tau_n)$, which is processed by the commitment algorithm of the HVC scheme to generate a compact fingerprint C. The user records the fingerprint C on the anonymous blockchain and uploads all ciphertexts $\boldsymbol{c} \leftarrow (c_1, \cdots, c_n)$ and tags $\boldsymbol{\tau} \leftarrow (\tau_1, \cdots, \tau_n)$ to the cloud server. To reduce the user's computation overhead, the server then computes a proof π_i for each tag τ_i using the opening algorithm of HVC, and stores the corresponding vectors of ciphertexts $\boldsymbol{c}$, tags $\boldsymbol{\tau}$, and proofs $\boldsymbol{\pi}$ accordingly.

When the user issues a SingleQuery for a data item m_i at index i, as illustrated in Fig. 5b, the server returns the corresponding ciphertext c_i, tag τ_i, and proof π_i. To ensure that the returned response is the latest (i.e., to detect server rollback attacks or malicious data tampering), the user first verifies the proof π_i of the tag τ_i against the fingerprint C using the verification algorithm of HVC. Once the verification succeeds, the user further checks the integrity of ciphertext c_i using tag τ_i. If valid, the user can recover the original data item m_i. Moreover, as illustrated in Fig. 5c, our reliable cloud synchronization mechanism supports BatchQuery, enabling the user to verify the reliability of a subset of datasets $\{m_i\}_{i \in I}$ ($I \subset [1, n]$). Instead of returning an individual proof π_i for each queried item m_i, the server aggregates them into a single proof π_I, thereby reducing the communication overhead. This aggregated proof π_I enables the user to uniformly verify the freshness of each queried data item m_i of the subset $I \subset [1, n]$, without relying on individual proofs π_i ($i \in I$).

To Update an old data item m_i^{old} to a new one m_i^{new}, as illustrated in Fig. 5d, the user first downloads m_i^{old} from the server and performs reliability verification. Upon successful verification, the user encrypts the new data item m_i^{new} to obtain the ciphertext-tag pair $(c_i^{new}, \tau_i^{new})$ and computes an incremental tag vector $\boldsymbol{\tau}_\delta$ reflecting the change in the tag vector

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on October 22,2026 at 21:15:32 UTC from IEEE Xplore. Restrictions apply.

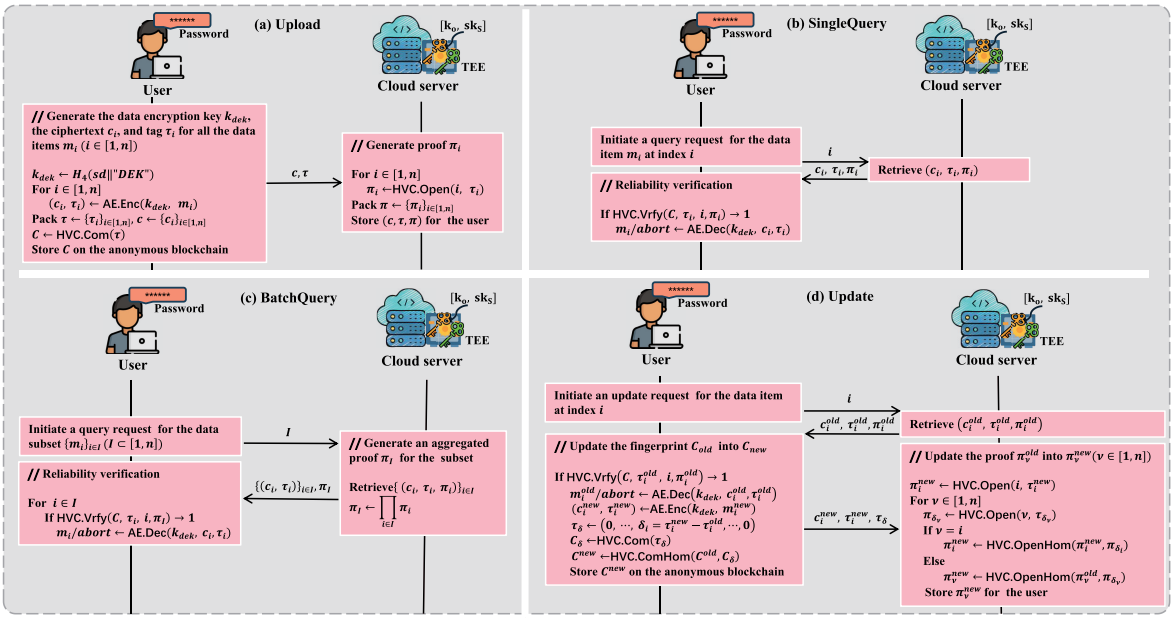


Fig. 5. Reliable cloud synchronization.

$\boldsymbol{\tau}$. Based on $\boldsymbol{\tau}_\delta$, the user derives an incremental fingerprint C_δ and applies it to update the old fingerprint C^{old} into a new one C^{new}. The user then sends the update entry $(\boldsymbol{\tau}_\delta, c_i^{new}, \tau_i^{new})$ to the server, which generates a new proof π_i^{new} for the new tag τ_i^{new} and uses the incremental tag vector $\boldsymbol{\tau}_\delta$ to update all existing proofs. Finally, the server stores the latest version of proofs $\boldsymbol{\pi}^{new}$.

In HRACS, data deletion is modeled as a special case of the update operation, in which the target data item is overwritten with a zero value. To delete m_i, the user first invokes SingleQuery to verify its existence. If the query result is zero, the deletion aborts. Otherwise, the client sets $m_i^{new} = 0$, generates the ciphertext-tag pair $(c_i^{new}, \tau_i^{new})$, and computes the incremental tag vector $\boldsymbol{\tau}_\delta \leftarrow (0, \cdots, \delta_i = \tau_i^{new} - \tau_i^{old}, \cdots, 0)$ along with the updated fingerprint C^{new}. Upon receiving the remove request, the cloud server replaces the old ciphertext c_i^{old} with the new ciphertext c_i^{new} and updates the proof vector $\boldsymbol{\pi}$. To verify compliance, the user can issue a post-deletion query. Since the server cannot forge valid proofs for an incorrect state, HRACS achieves verifiable deletion. Notably, such a mechanism only guarantees logical deletion, while verifiable physical deletion (e.g., removal from all storage media or backups) cannot be guaranteed through cryptographic mechanisms. However, because all data is encrypted under a user-generated data encryption key k_{dek}, any residual ciphertext remains confidential to the server. Consequently, retaining obsolete ciphertext yields no functional benefit but incurs maintenance costs, making it economically irrational for service providers to preserve deleted data in reality.

V. Security Analysis

The security definition and proof of our HRACS scheme are provided in the ROM, where hash functions are modeled as random oracles. Moreover, the PPT adversary $\mathcal{A}$ is granted access to the following oracles:

Adversarial Oracles

- Reg$O(M)$. Given a message M, a user $\mathcal{U}$ whose $(\rho, pk_{\mathcal{U}}, \sigma_\rho)$ are uninitialized executes the registration protocol and returns the resulting $(\rho, pk_{\mathcal{U}}, \sigma_\rho)$ to $\mathcal{A}$. This oracle captures $\mathcal{A}$'s ability to initiate the registration process.
- Send$O_1(M)$. Given a message M, $\mathcal{U}$ and $\mathcal{CS}$ engage in the protocol and return the transcript to $\mathcal{A}$. This oracle captures $\mathcal{A}$'s ability to observe all users.
- Send$O_2(M)$. Given a message M, a user $\mathcal{U}$ whose $(\rho, pk_{\mathcal{U}}, \sigma_\rho)$ are initialized and $\mathcal{CS}$ execute the protocol and return the results to $\mathcal{A}$. This oracle captures $\mathcal{A}$'s ability to observe registered users.
- Execute$O(\mathcal{U}, \mathcal{CS})$. This oracle triggers an honest execution between $\mathcal{U}$ and $\mathcal{CS}$ and returns the transcript to $\mathcal{A}$. It captures the ability to observe honest executions and is essential for modeling offline PGA resistance.
- Reveal$O(\mathcal{U}, \mathcal{CS})$. This oracle returns the previously established session key between $\mathcal{U}$ and $\mathcal{CS}$.
- Store$O(\boldsymbol{m})$. Given a dataset $\boldsymbol{m}$, this oracle returns $(C, \boldsymbol{m}, \boldsymbol{c}, \boldsymbol{\tau}, \boldsymbol{\pi})$ to $\mathcal{A}$, modeling the ability to initiate a data deposit to $\mathcal{CS}$.
- DataUpdate$O(C, i, m_i^{new})$. Given a dataset fingerprint C, an index $i \in [1,n]$, and a new item m_i^{new}, this oracle returns $(C^{new}, m_i^{new}, c_i^{new}, \tau_i^{new}, \pi_i^{new})$ to $\mathcal{A}$, capturing the ability to update stored data.

Based on the above oracles and experiments illustrated in Fig. 6, we formalize the security definition of our HRACS scheme in terms of *AKE security*, *portability*, *anonymity*, *confidentiality*, and *reliability* below:

Definition 1 (AKE security): A scheme is said to be AKE secure if for any PPT adversary $\mathcal{A}$, there exists a negligible function negl such that

$$Pr[\mathsf{Exp}^{\mathsf{AKE}}_{\mathsf{HRACS},\mathcal{A}}(\lambda) \rightarrow 1] \leq \frac{1}{2} + \frac{q_{se}}{|\mathcal{PW}|} + \mathsf{negl}(\lambda), \quad (1)$$

where q_{se} is the upper bound to query Send$O_1(M)$.

Theorem 1: If DS is EUF-CMA secure, OPRF is pseudo-random, and all the hash functions can be modeled as random oracles, our HRACS is AKE secure.

Proof: Due to space constraints, the proof of Theorem 1 is deferred to Appendix A-A (see Supplementary Material). ■

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on October 22,2026 at 21:15:32 UTC from IEEE Xplore. Restrictions apply.

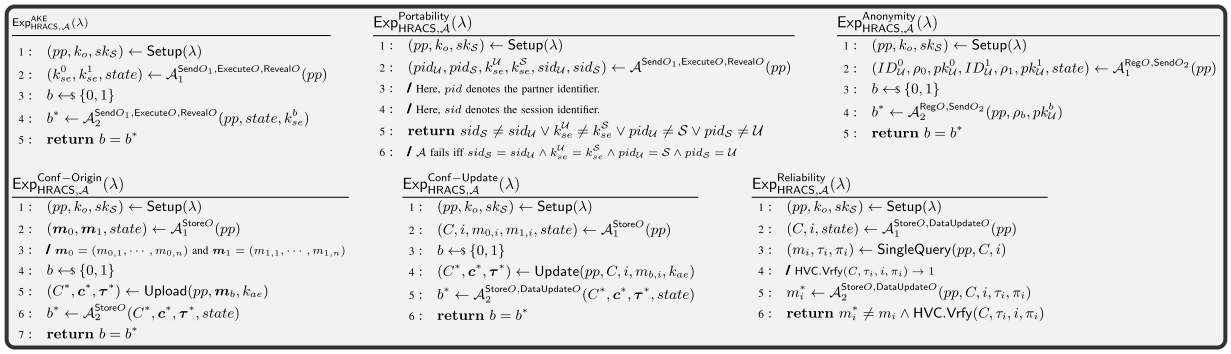


Fig. 6. Security experiments of our HRACS.

Definition 2 (Portability): A scheme is said to be portable if for any PPT adversary $\mathcal{A}$, there exists a negligible function negl such that

$$Pr[\mathsf{Exp}^{\mathsf{Portability}}_{\mathsf{HRACS},\mathcal{A}}(\lambda) \rightarrow 1] \leq \frac{q_{se}}{|\mathcal{PW}|} + \mathsf{negl}(\lambda), \tag{2}$$

where q_{se} is the upper bound to query $\mathsf{Send}O_1(M)$.

Theorem 2: If DS is EUF-CMA secure, OPRF is pseudorandom, and all the hash functions can be modeled as random oracles, HRACS achieves portability.

Proof: Due to space constraints, the proof of Theorem 2 is deferred to Appendix A-B (see Supplementary Material). ■

Definition 3 (Anonymity): A scheme is said to be anonymous if for any PPT adversary $\mathcal{A}$, there exists a negligible function negl such that

$$Pr[\mathsf{Exp}^{\mathsf{Anonymity}}_{\mathsf{HRACS},\mathcal{A}}(\lambda) \rightarrow 1] \leq \frac{1}{2} + \frac{q_{se}}{|\mathcal{PW}|} + \mathsf{negl}(\lambda). \tag{3}$$

Theorem 3: If DS is EUF-CMA secure, OPRF is pseudorandom, and all the hash functions can be modeled as random oracles, HRACS achieves anonymity.

Proof: Due to space constraints, the proof of Theorem 3 is deferred to Appendix A-C (see Supplementary Material). ■

Definition 4 (Confidentiality): HRACS achieves confidentiality if for any PPT adversary $\mathcal{A}$, there exists a negligible function negl such that

$$Pr[\mathsf{Exp}^{\mathsf{Conf-Origin}}_{\mathsf{HRACS},\mathcal{A}}(\lambda) \rightarrow 1] \leq \frac{1}{2} + \frac{q_{se}}{|\mathcal{PW}|} + \mathsf{negl}(\lambda), \tag{4}$$

and

$$Pr[\mathsf{Exp}^{\mathsf{Conf-Update}}_{\mathsf{HRACS},\mathcal{A}}(\lambda) \rightarrow 1] \leq \frac{1}{2} + \frac{q_{se}}{|\mathcal{PW}|} + \mathsf{negl}(\lambda), \tag{5}$$

where q_{se} is the upper bound to query $\mathsf{Send}O_1(M)$.

Theorem 4: If OPRF is pseudorandom and AE is IND-CCA secure, then our HRACS achieves confidentiality.

Proof: Due to space constraints, the proof of Theorem 4 is deferred to Appendix A-D (see Supplementary Material). ■

Definition 5 (Reliability): HRACS achieves reliability if for any PPT adversary $\mathcal{A}$, there exists a negligible function negl such that

$$Pr[\mathsf{Exp}^{\mathsf{Reliability}}_{\mathsf{HRACS},\mathcal{A}}(\lambda) \rightarrow 1] \leq \frac{q_{se}}{|\mathcal{PW}|} + \mathsf{negl}(\lambda), \tag{6}$$

where q_{se} is the upper bound to query $\mathsf{Send}O_1(M)$.

TABLE V
CRYPTOGRAPHIC CONFIGURATION (λ = 128)

Primitive	Instantiation	Parameter (Bit)†
HVC‡	CDH-based HVC	N/A
OPRF‡	2HashDH	$\lvert k_o\rvert = 256$
AE	AES-GCM	$\lvert k_{dek}\rvert = 256$
Hash	SHA-256	$\lvert\rho\rvert = 144$, $\lvert k_{se}\rvert = 256$

† Bit lengths of critical parameters. ‡ The primitives are instantiated on the Type A curve, defined by $y^2 = x^3 + x$. N/A denotes that the corresponding entry is not involved.

Theorem 5: If AE is INT-CTXT secure, OPRF is pseudorandom, and VC is position-binding, HRACS achieves reliability.

Proof: Due to space constraints, the proof of Theorem 5 is deferred to Appendix A-E (see Supplementary Material). ■

VI. Performance Evaluation

In this section, we first describe the experimental setup of our proposed HRACS. We then evaluate the robustness of our PIR-free APAKE protocol and analyze its computation and communication overhead in comparison with traditional APAKE schemes [16], [18], [19]. Building on the formal security analysis presented in Section V, which establishes that HRACS provides both anonymity and reliability, we further benchmark its performance in terms of computation and communication overhead against existing PBCS schemes (e.g., PHE [4], PLCS [58],[10] and IPBCS [68]). In addition, we evaluate the performance of anonymous blockchain. Overall, the results demonstrate the practical efficiency of HRACS.

A. Experimental Setup

1) Implementation Environment: HRACS is evaluated using a client–server (CS) architecture. The client runs on a virtual machine with Ubuntu 20.04.2 LTS, 2 CPU cores, and 8 GB of RAM. The virtual machine is hosted on a local machine equipped with an Intel Core i7-9750H CPU (2.60 GHz) and 16 GB of RAM, running Windows 11. The server is deployed on a physically separate remote machine, and the client communicates with it over a real network

[10]To avoid confusion with the PBCS terminology in this paper, the original PBCS scheme (Portable Blind Cloud Storage) is renamed as PLCS.

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on October 22,2026 at 21:15:32 UTC from IEEE Xplore. Restrictions apply.

using TCP/IP sockets. HRACS is implemented in C++17 and compiled with g++ 11.4.0 using CMake 3.22.1. Cryptographic functions are provided by PBC 0.5.14[11] and OpenSSL 3.0.2.[12] With security parameter $\lambda = 128$, the cryptographic configuration of HRACS is summarized in Table V.

To evaluate the performance of the anonymous blockchain, we implemented a smart contract in Solidity 0.8.20 and compiled and deployed it through the Remix IDE.[13] The contract is deployed on the Oasis Sapphire Testnet (Chain ID: 23295),[14] while network interactions and transaction signing are handled via the MetaMask wallet.[15] We select Oasis Sapphire because native anonymous blockchains (e.g., Monero[16] and Zcash[17]) provide strong transaction anonymity but do not support smart contracts required for dynamic read/write (i.e., retrieval/storage) operations. In contrast, Oasis Sapphire is a TEE-backed confidential blockchain that offers end-to-end encryption for both transactions and queries while supporting full smart-contract programmability. Since contract execution occurs inside enclaves, sensitive data remains protected from external observers, reducing the risk of identity and data linkage while preserving the flexibility of Web3 applications.

2) Dataset: For robustness evaluation, we use four password datasets,[18] each containing 20,000 user passwords. The first consists of 6-digit passwords, simulating real-world scenarios such as payment PINs. The second dataset consists of 12-character passwords, including letters, digits, and special symbols. These passwords are derived from a combination of the *RockYou* dataset (a widely used collection of real-world leaked passwords for password security research),[19] user-specific public information (e.g., name, birthday, and phone number), and additional deterministic randomization (e.g., *mangling rules*[20]). The third dataset contains passwords with lengths uniformly ranging from 6 $\sim$ 12 characters, reflecting a more realistic password length distribution. Finally, we construct a passphrase dataset derived from the *EFF Large Wordlist* (a dataset commonly used to generate human-memorable passphrases).[21] For HRACS evaluation, we use a dataset of 600 individual `.dat` files, each 1 MB.[22]

3) Baseline: To assess the computation efficiency of our PIR-free APAKE, we compare it with two representative APAKE protocols [18], [19]. For communication efficiency, we employ a PIR-based APAKE protocol [16] as the baseline. To assess HRACS, we benchmark it against three state-of-the-art PBCS schemes: PHE [4], PLCS [58], and IPBCS [68].

4) Metrics: To evaluate the robustness of our PIR-free APAKE against online PGA, we use the *attack success rate* ASR, quantifying the fraction of users whose passwords are successfully compromised by an adversary given a fixed number of online guesses. Specifically, $ASR(n_g) = \frac{C(n_g)}{N}$, where N is the total number of users, and $C(n_g)$ is the number of accounts compromised after n_g online guesses. Note that a lower $ASR(n_g)$ indicates higher protocol robustness.

To evaluate the computation efficiency, we define the *computation overhead* $Comp_{i,j}$ as the total execution time (in milliseconds or seconds) consumed by scheme i when executing algorithm j. Formally, $Comp_{i,j} = \sum_{\ell=1}^{o_{i,j}} t_\ell$, where t_ℓ denotes the execution time of the ℓ-th computation step, and $o_{i,j}$ represents the number of computation operations required by scheme i when performing algorithm j.

To measure the communication efficiency, we define the *communication overhead* $Comm_{i,j}$ as the total amount of data transmitted (in kilobytes or megabytes) by scheme i during algorithm j. Formally, $Comm_{i,j} = \sum_{\ell=1}^{r_{i,j}} |m_\ell|$, where m_ℓ denotes the ℓ-th message exchanged and $r_{i,j}$ is the number of message rounds of scheme i under algorithm j.

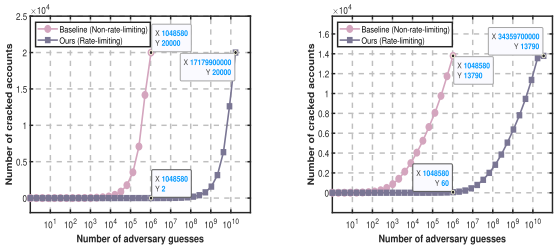
To evaluate the economic viability of the blockchain, we define the *total network cost* $TNC = N \times f \times Cost$ to capture the aggregate financial overhead under large-scale deployment, where N denotes the number of active users, and f represents the average frequency of on-chain fingerprint update operations performed by each user per day. Crucially, the unit on-chain transaction cost is formulated as $Cost = Gas \times GasPrice \times 10^{-9} \times V_{token}$, where Gas denotes the gas consumption incurred by invoking the smart contract functions, and $GasPrice$ represents the network gas price (in Gwei). Based on the real-time market data retrieved from CoinMarketCap,[23] the market price of the native utility token is calibrated as $V_{token} = 0.006$ USD. A higher TNC reflects increased financial expenditure under large-scale utilization.

Furthermore, to evaluate the economic sustainability and scalability of HRACS, we define the *projected transaction workload* $WL = \frac{N \times f}{86{,}400}$ to measure the expected number of transaction requests generated per second (1 day = 86,400 seconds). A higher WL simulates a more intensive concurrent update environment, thereby testing the boundary scalability of the system. In this context, if the system maintains a low amortized cost even under a high WL without exceeding the network's maximum throughput (TPS), it demonstrates robust scalability and long-term sustainability.

5) Code Availability: To ensure transparency, reproducibility, and fair comparison, we independently re-implemented all baseline schemes and our proposed HRACS framework. The released source code includes online PGA,[24] APAKE baselines,[25,26] PBCS baselines,[27] and our proposed HRACS.[28] In

[11] https://github.com/blynn/pbc
[12] https://github.com/openssl/openssl
[13] https://remix.ethereum.org/
[14] https://thirdweb.com/oasis-sapphire-testnet
[15] https://metamask.io/
[16] https://www.getmonero.org/
[17] https://z.cash/
[18] https://www.kaggle.com/datasets/anonymous4open/password-corpora
[19] https://weakpass.com/wordlists/rockyou.txt
[20] Mangling rules transform high-probability passwords into various modified forms, such as changing letter capitalization, substituting digits or symbols, or adding prefixes or suffixes. These transformations significantly expand the set of plausible guesses without requiring exhaustive enumeration.
[21] https://www.eff.org/document/passphrase-wordlists
[22] https://www.kaggle.com/datasets/anonymous4open/benchmark-dataset
[23] https://coinmarketcap.com/currencies/oasis-network/
[24] https://github.com/TommyLZ/HRACS_Articraft/tree/main/OnlinePGA
[25] https://github.com/TommyLZ/HRACS_Articraft/tree/main/APAKE_Benchmark
[26] https://github.com/TommyLZ/HRACS_Articraft/tree/main/APAKE_Communication_Test
[27] https://github.com/TommyLZ/HRACS_Articraft/tree/main/PBCS_Benchmark
[28] https://github.com/TommyLZ/HRACS_Articraft/tree/main/HRACS

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on October 22,2026 at 21:15:32 UTC from IEEE Xplore. Restrictions apply.



(a) 6-digit passwords. (b) 12-character passwords.

(c) 6∼12-character passwords. (d) 3∼6-word passphrase.

Fig. 7. Robustness evaluation of our PIR-free APAKE against online PGA for up to 20,000 users in terms of (a) 6-digit passwords, (b) 12-character passwords (containing uppercase and lowercase letters, digits, and symbols), (c) 6 ∼ 12-character passwords, and (d) 3 ∼ 6-word passphrase, evaluated under the baseline (non-rate-limiting) and rate-limiting settings.

addition, the Solidity implementation used for the anonymous blockchain evaluation is publicly available.[29]

B. Performance Evaluation of PIR-Free APAKE

1) Robustness Evaluation: Under the assumptions in Section III-D, the adversary can launch an online PGA against our PIR-free APAKE protocol. Specifically, targeting a user with public identity $ID_{\mathcal{U}}$, the adversary first constructs a list of candidate passwords $\{\widetilde{pw}_1, \widetilde{pw}_2, \ldots\}$, either by exhaustively enumerating the password space or applying mangling rules.

Let the user's actual password be denoted by $pw^{\star}$. For each candidate password $\widetilde{pw}$, the adversary performs an online login attempt using $(ID_{\mathcal{U}}, \widetilde{pw})$ and collects the server's response, containing the value $\mathcal{H}(ID_{\mathcal{U}} \,\|\, \widetilde{pw})^{k_o}$. Using this value, the adversary can compute the candidate pseudonym

$$\rho_{\widetilde{pw}} = H_1\big(ID_{\mathcal{U}} \,\|\, \mathsf{H}(\widetilde{pw} \,\|\, \mathcal{H}(ID_{\mathcal{U}} \,\|\, \widetilde{pw})^{k_o})\big),$$

and compares it with the pseudonym entries recorded in the compromised registration table. Once a match is found (i.e., $\rho_{\widetilde{pw}} = \rho$), the adversary concludes that $\widetilde{pw} = pw^{\star}$. As a result, the pseudonym ρ is linked to the public identity $ID_{\mathcal{U}}$, compromising the anonymity expected by the protocol.

Using the above methodology, we evaluate the robustness of our PIR-free APAKE against online PGA under both baseline (non-rate-limiting) and rate-limiting settings, as illustrated in Fig. 7. In our rate-limiting design, each pseudonym is allowed up to five consecutive failed authentication attempts. Once this threshold is exceeded, the system enforces a cooldown period of 5 min during which all further guesses associated with the same pseudonym are rejected. Given that one authentication attempt in our deployed environment requires approximately 3.26 ms, a 5 min cooldown corresponds to at most $\frac{300\,\text{s}}{0.00326\,\text{s}} \approx$ 92,024 attempts that are blocked. Our goal is to quantify how per-pseudonym rate-limiting constrains an adversary's online guessing capability and consequently reduces the number of compromised users out of 20,000.

For online PGA on the 6-digit password dataset illustrated in Fig. 7a, we exhaustively enumerate the entire password space for 20,000 users. Under the baseline setting, an adversary performing 1,048,580 online guesses can fully compromise all users. In contrast, with pseudonymous rate-limiting enabled, the same number of guesses compromises only 2 users. This corresponds to a reduction in attack success rate (i.e., ASR) from 100% to 0.01% (a four-order-of-magnitude drop), effectively demonstrating that rate-limiting almost completely suppresses large-scale compromise despite identical attacker effort. This effect becomes even clearer when considering scalability: fully compromising all users under rate-limiting would require 17,179,900,000 online attempts.

For online PGA on the 12-character (i.e., containing lowercase and uppercase letters, digits, and symbols) password dataset illustrated in Fig. 7b, exhaustive enumeration of the full space is computationally infeasible. To produce realistic password guesses, we adopt mangling rules to generate candidate passwords. In the baseline configuration, 1,048,580 mangled guesses successfully compromise 13,790 users.[30] When pseudonymous rate-limiting is enabled, the same number of guesses compromises only 60 users, decreasing ASR from 68.95% to 0.30%.[31] Achieving the same compromise level of 13,790 users in the rate-limited setting would instead require 34,359,700,000 online attempts, demonstrating the significant robustness improvement introduced by rate-limiting.

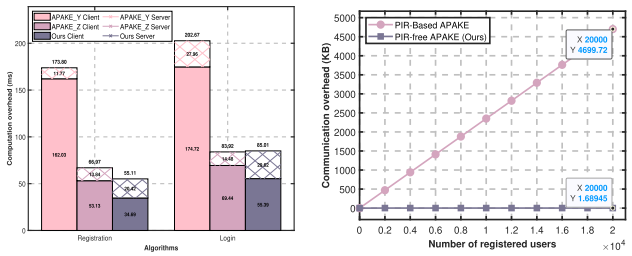
For online PGA on the 6 ∼ 12-character password dataset illustrated in Fig. 7c, we observe a realistic guessing scenario based on mangling rules over leaked password distributions. Under the baseline setting, 1,048,580 generated guesses successfully compromise 4,558 users. When pseudonymous rate-limiting is enabled, the same number of guesses only compromises 53 users, reducing the ASR from 22.79% to 0.27%. To reach the same compromise level of 4,558 users under rate-limiting, the adversary would require approximately 34,359,700,000 online attempts. For online PGA on the 3 ∼ 6-word passphrase dataset, as illustrated in Fig. 7d, under the baseline, 1,048,580 guesses compromise 13,744 accounts. With rate-limiting, the number of compromised accounts drops to 60, reducing the ASR from 68.72% to 0.30%. To reach the same compromise level as the baseline, the adversary would also require approximately 34,359,700,000 attempts.

[29]https://github.com/TommyLZ/HRACS_Articraft/tree/main/Blockchain_IO_Test

[30]The 12-character password space is prohibitively large, rendering exhaustive enumeration computationally infeasible. Consequently, the adversary cannot achieve full user coverage as with 6-digit passwords and is instead limited to high-probability guesses.

[31]The 12-character dataset contains real leaked passwords with strong statistical bias. This bias amplifies the effect of mangling rules, generating many high-probability guesses that match common user patterns. As a result, adversaries can often find correct passwords much earlier, making the attack more effective than in the 6-digit dataset, where no such improvement appears.

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on October 22,2026 at 21:15:32 UTC from IEEE Xplore. Restrictions apply.



(a) Computation overhead. (b) Communication overhead.

Fig. 8. Comparison between traditional APAKE schemes [16], [18], [19] and our PIR-free APAKE in terms of (a) client-side, server-side, and total computation overhead (with APAKE$_Y$ [18] and APAKE$_Z$ [19]), and (b) communication overhead for up to 20,000 users (with PIR-based APAKE [16] instantiated with OPRF).

Overall, the results across these password types demonstrate a consistent trend: with pseudonymous rate-limiting enabled, the number of compromised accounts under any fixed guessing budget drops sharply, and the required effort for large-scale compromise increases from millions to tens of billions of attempts. In real-world deployments, exceeding the five-attempt threshold typically triggers additional safeguards, such as temporary account lockout and security notifications (via SMS or e-mail) to alert users and prompt timely password updates. We further stress that extremely weak passwords (e.g., "000000" or "123456") are strongly discouraged in practice, and most systems explicitly forbid such patterns during password creation. These combined mechanisms reinforce the robustness of our PIR-free APAKE.

2) Computation Overhead: As illustrated in Fig. 8a, we compare the total computation overhead (i.e., the sum of the client-side and server-side costs) of our PIR-free APAKE against two baselines, APAKE$_Y$ [18] and APAKE$_Z$ [19], with 20,000 registered users. For Registration, our scheme requires only 55.11 ms, reducing the overhead by 68.29% and 17.71% compared with APAKE$_Y$ (173.80 ms) and APAKE$_Z$ (66.97 ms), respectively. For Login, our scheme incurs 85.01 ms, achieving a 58.05% reduction over APAKE$_Y$ (202.67 ms) and performance comparable to APAKE$_Z$ (83.92 ms). These results demonstrate that our design provides competitive computation efficiency.

To further explain the computation efficiency observed above, Table VI illustrates a fine-grained comparison of the dominant cryptographic operations, namely modular exponentiations and bilinear pairings. The corresponding unit-test source code is publicly available.[32] Compared with APAKE$_Y$, our PIR-free APAKE reduces the number of modular exponentiations from 29 to 11 and completely eliminates the 6 bilinear pairings, decreasing the estimated computation overhead from 117.37 ms to 34.87 ms. Compared with APAKE$_Z$, our scheme requires two additional modular exponentiations (11 versus 9). These results are consistent with Fig. 8a, showing that our PIR-free APAKE achieves substantially lower computation overhead than APAKE$_Y$ and remains comparable to APAKE$_Z$.

[32]https://github.com/TommyLZ/HRACS_Articraft/tree/main/Operation_UnitTest

TABLE VI

FINE-GRAINED COMPARISON OF CRYPTOGRAPHIC OPERATIONS BETWEEN TRADITIONAL APAKE SCHEMES [18], [19] AND OUR PIR-FREE APAKE SCHEME.[†]

Operation	**APAKE$_Y$** [18]	**APAKE$_Z$** [19]	**PIR-free APAKE**
Exponentiation	29	9	11
Pairing	6	0	0
Summary (ms)	**117.37**	**28.53**	**34.87**

[†] The average execution times of a modular exponentiation and a bilinear pairing, measured over 1,000 runs, were 3.17 ms and 4.24 ms, respectively.

Overall, the substantial reduction in computation overhead relative to APAKE$_Y$ mainly stems from the decreased exponentiation workload and the complete elimination of bilinear pairings. This improvement can be further explained by the constructions of the baseline schemes. APAKE$_Y$ relies on computationally intensive building blocks, including ZKP [21] (i.e., exponentiations dominated) and HE [22] (i.e., pairing dominated), and APAKE$_Z$ improves efficiency by replacing HE with a lightweight algebraic MAC [38]. In contrast, our PIR-free APAKE eliminates these expensive primitives and employs a lightweight construction with no bilinear pairings. Consequently, it substantially outperforms APAKE$_Y$ while maintaining computation efficiency comparable to APAKE$_Z$.

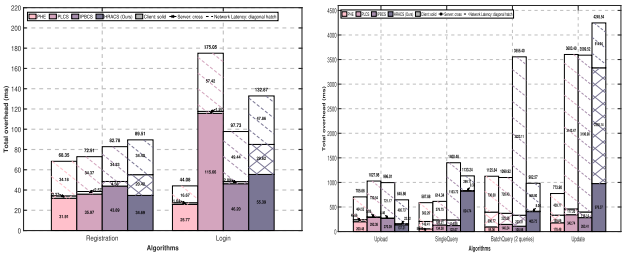
3) Communication Overhead: As illustrated in Fig. 8b, using the *Comm* metric, we evaluate the communication overhead by comparing our PIR-free APAKE with the PIR-based APAKE [16] instantiated with OPRF, as the number of registered users increases from 2,000 to 20,000. The results show that the PIR-based construction exhibits an approximately linear increase in communication overhead, reaching 4,699.72 KB at 20,000 registered users. In contrast, our PIR-free APAKE consistently maintains an almost constant communication cost of approximately 1.69 KB, indicating that its communication overhead is independent of the number of registered users. Notably, at 20,000 users, our scheme reduces the communication overhead by approximately 99.96% compared with the PIR-based construction.

Overall, our PIR-free APAKE simultaneously achieves high robustness and low computation and communication overhead.

C. Performance Evaluation of HRACS

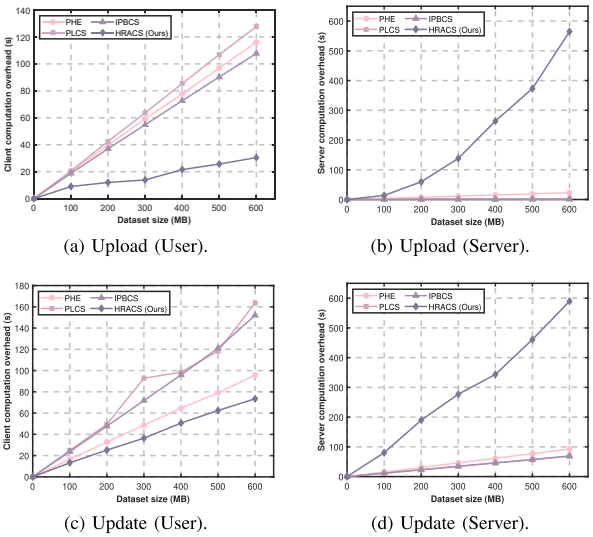
1) Computation Overhead: Fig. 9 compares the computation overhead of PHE [4], PLCS [58], IPBCS [68], and our HRACS across six operations, with each result decomposed into client-side computation, server-side computation, and network latency. HRACS achieves the lowest total overhead for Upload and BatchQuery, requiring 645.86 ms and 982.57 ms, respectively, where the BatchQuery result is normalized to retrieving two files. Regarding transmission latency, all four schemes exhibit comparable performance during Registration, with values ranging from 34.18 ms to 34.40 ms. HRACS achieves the lowest network latency for SingleQuery and BatchQuery, at 299.17 ms and 568.80 ms, respectively, while attaining the second-lowest latency for Login, Upload, and Update. In particular, its Update latency remains substantially lower than those of PLCS and IPBCS. These results demonstrate that our HRACS achieves competitive computation

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on October 22,2026 at 21:15:32 UTC from IEEE Xplore. Restrictions apply.



(a) Registration & Login. (b) Data-intensive algorithms.

Fig. 9. Computation overhead comparison among PHE [4], PLCS [58], IPBCS [68], and our HRACS, evaluated on a dataset of 600 files (1 MB each). Each bar shows the network latency, server-side, and client-side computation overhead, with the total overhead labeled above the bar in terms of (a) Registration & Login, and (b) data-intensive algorithms, where Upload and SingleQuery each process 1 file, BatchQuery retrieves 2 files, and Update modifies 1 file.



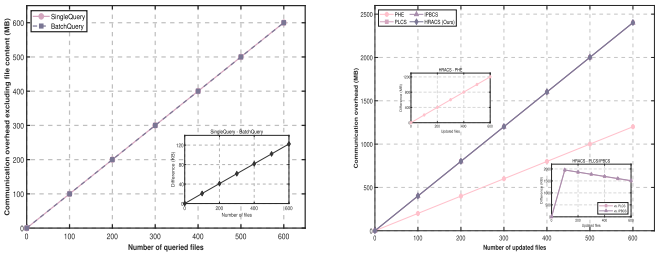
(a) Upload (User). (b) Upload (Server).

(c) Update (User). (d) Update (Server).

Fig. 10. Comparison of the computation overhead among PHE [4], PLCS [58], IPBCS [68], and our HRACS in terms of (a) user-side Upload, (b) server-side Upload, (c) user-side Update, and (d) server-side Update, evaluated on a dataset of 600 files (1 MB each).

efficiency and favorable transmission latency, particularly for data-intensive algorithms.

In addition, we analyze how the computation overhead scales with the dataset size. As illustrated in Fig. 10, the user-side and server-side costs of both Upload and Update generally increase with the number of files. For user-side Upload, HRACS consistently achieves the lowest overhead and requires 30.43 s for 600 files, compared with 116.10 s for PHE, 127.63 s for PLCS, and 107.80 s for IPBCS. This corresponds to an average client-side cost of approximately 50.72 ms per file. The server-side Upload overhead of HRACS reaches 564.76 s for 600 files, compared with 22.82 s for PHE, 1.42 s for PLCS, and 1.52 s for IPBCS. A similar trade-off is observed for Update. On the user side, HRACS again achieves the lowest overhead, requiring 73.48 s for updating 600 files, whereas PHE, PLCS, and IPBCS require 95.76 s, 163.56 s, and 151.99 s, respectively. On the server side, HRACS requires 589.67 s, compared with 92.51 s for PHE, 68.62 s for PLCS, and 69.05 s for IPBCS. In practical cloud deployments, servers are typically provisioned with substantially greater computational resources than user devices and can further exploit



(a) Single vs. Batch query. (b) Update.

Fig. 11. Communication scalability for datasets containing up to 600 files (1 MB each): (a) comparison between SingleQuery and BatchQuery in HRACS and (b) Update communication overhead of PHE [4], PLCS [58], IPBCS [68], and our HRACS.

parallel processing or horizontal scaling to accommodate such workloads. Therefore, shifting additional cryptographic processing to the resource-rich server reduces the burden on comparatively resource-constrained clients. These results demonstrate that HRACS prioritizes user-side efficiency for data-intensive operations, thereby improving practical user experience while supporting datasets containing up to 600 files.

2) Communication Overhead: As illustrated in Fig. 11a, both SingleQuery and BatchQuery grow approximately linearly with the number of queried files and exhibit similar overheads because both transmit ciphertexts and authentication information. Nevertheless, BatchQuery consistently reduces communication overhead through proof aggregation. When querying 600 files, repeated SingleQuery executions incur approximately 600.20 MB of overhead, whereas BatchQuery requires only 600.08 MB. As illustrated in the inset, the communication savings achieved by BatchQuery increase from 20.63 KB for 100 files to 122.32 KB for 600 files, demonstrating that its advantage becomes increasingly pronounced as the number of queried files grows. Fig. 11b compares the Update communication overhead across the four schemes. For 600 updated files, PHE, PLCS, IPBCS, and HRACS transmit approximately 1200.15 MB, 2400.83 MB, 2400.82 MB, and 2402.30 MB, respectively. Although HRACS incurs 1202.15 MB more communication than PHE, its overhead is close to those of the structurally similar PLCS and IPBCS schemes, exceeding them by only 1501.59 KB and 1511.67 KB, respectively. This modest increase arises from the additional proof and update-authentication information required to support reliable cloud synchronization.

Overall, the communication overhead of HRACS scales approximately linearly with the number of operated files and remains comparable to PLCS and IPBCS. These results demonstrate that HRACS provides stronger synchronization and integrity guarantees with only limited additional communication relative to closely related PBCS schemes.

D. Performance of Anonymous Blockchain

Table VII illustrates the average performance of anonymous blockchain-based pseudonym and fingerprint storage and retrieval over 15 consecutive runs. On average, an on-chain storage operation incurs an RPC latency of 4.41 s and a confirmation latency of 12.78 s, whereas a retrieval

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on October 22,2026 at 21:15:32 UTC from IEEE Xplore. Restrictions apply.

TABLE VII

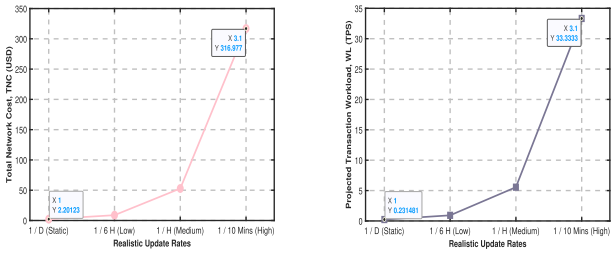
AVERAGE PERFORMANCE OF ANONYMOUS BLOCKCHAIN STORAGE AND RETRIEVAL OPERATIONS OVER 15 CONSECUTIVE RUNS.†

Storage				Retrieval
RPC latency (s)	**Confirmation latency** (s)	**Gas**	**Cost** (10^{-4} USD)	**RPC latency** (s)
4.41	12.78	183436	1.10	0.34

† **RPC latency:** End-to-end delay of request transmission and node response. **Confirmation latency:** Time elapsed from transaction submission to blockchain confirmation. **Gas:** Computation overhead incurred by smart contract execution. The gas price was fixed at 100 Gwei for all storage operations. **Cost:** Monetary cost of a blockchain storage operation, calculated as

$$\textbf{Cost}\ (\text{USD}) = \textbf{Gas} \times \textbf{Gas Price}\ (\text{Gwei}) \times \mathbf{10^{-9}} \times \mathbf{V}_{\text{token}}.$$

Here, $\mathbf{V}_{\text{token}} = 0.006$ USD/ROSE denotes the USD value of one ROSE token.



(a) Total network cost. (b) Projected transaction workload.

Fig. 12. Cost and throughput analysis of the anonymous blockchain system under 20,000 active users for (a) total network cost and (b) projected transaction workload.

operation achieves an average RPC latency of only 0.34 s. Such performance characteristics are well suited to practical deployment, since the relatively higher storage latency is confined to infrequent user registration or update operations, whereas latency-critical login and update verification rely solely on low-latency retrieval operations. Moreover, each storage transaction consumes an average of 183,436 gas. Given a gas price of 100 Gwei and a token value of 0.006 USD/ROSE, the average monetary cost per storage operation is only approximately 0.00011 USD, imposing a negligible financial burden on users. Detailed results for all 15 runs are provided in Appendix B (see Supplementary Material).

Moreover, as illustrated in Fig. 12, we evaluate the economic viability and sustainability of the anonymous blockchain under four representative fingerprint update frequencies: static (once per day), low-frequency (once every 6 hours), medium-frequency (once per hour), and high-frequency (once every 10 minutes). We consider a large-scale HRACS deployment with 20,000 active users and evaluate *TNC* and *WL*. Fig. 12a illustrates the *TNC* at different update frequencies. Because the number of on-chain transactions is determined by both the number of users and the per-user update rate, the aggregate financial overhead grows linearly with the update frequency f. Specifically, increasing the update rate from once per day ($f = 1$) to once every 10 minutes ($f = 144$) raises the daily network cost by 144×, from approximately 2.2 USD to 316.98 USD. Even under the most intensive setting, this cost remains modest for 20,000 users, primarily owing to the low native-token valuation and the lightweight PIR-free APAKE design. Moreover, as illustrated in Fig. 12b, the throughput required by *WL* also grows linearly with the update frequency, increasing from approximately 0.23 TPS in the static setting to 33.33 TPS in the high-frequency setting. This demonstrates predictable scaling as the update interval becomes shorter.

Overall, the above results reveal a clear trade-off between update responsiveness and system overhead. More frequent fingerprint updates improve data freshness but increase both economic cost and transaction workload. Nevertheless, even under the most demanding configuration, HRACS maintains a modest deployment cost and a practical throughput requirement, demonstrating its economic viability and sustainability for large-scale real-world deployment.

VII. CONCLUSION

In this paper, we propose HRACS, a highly reliable anonymous cloud synchronization scheme. HRACS is built upon a novel PIR-free APAKE protocol that employs a pseudonym–signature binding mechanism, eliminating expensive cryptographic primitives and avoiding PIR-based approaches whose communication overhead grows linearly with the number of registered users. This achieves low computation and communication costs. The pseudonym further supports anonymous rate-limiting, thereby resisting online PGA and enhancing robustness. Moreover, HRACS simultaneously guarantees cloud synchronization reliability by generating a dataset fingerprint, which allows users to detect unauthorized data modifications. Experimental results demonstrate that HRACS achieves computation and communication overhead comparable to state-of-the-art schemes.

REFERENCES

[1] Fortune Business Insights. *Cloud Storage Market Size, Share & Industry Analysis*. Accessed: Sep. 24, 2026. [Online]. Available: https://www.fortunebusinessinsights.com/cloud-storage-market-102773

[2] Google. *Google Drive Help*. Accessed: Sep. 24, 2026. [Online]. Available: https://support.google.com/drive?sjid=5619061132282085809-NC#topic=14940

[3] Apple. *iCloud Support*. Accessed: Sep. 24, 2026. [Online]. Available: https://support.apple.com/icloud

[4] R. W. Lai, C. Egger, M. Reinert, S. S. Chow, M. Maffei, and D. Schröder, "Simple password-hardened encryption services," in *Proc. USENIX Secur.*, 2018, pp. 1405–1421.

[5] J. Brost, C. Egger, R. W. F. Lai, F. Schmid, D. Schröder, and M. Zoppelt, "Threshold password-hardened encryption services," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2020, pp. 409–424.

[6] S. Jarecki, H. Krawczyk, and J. Resch, "Updatable oblivious key management for storage systems," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2019, pp. 379–393.

[7] L. Chen, H. Guo, Y.-N. Li, and Q. Tang, "Efficient secure storage with version control and key rotation," in *Proc. ASIACRYPT*, 2023, pp. 168–198.

[8] M. Backendal, H. Davis, F. Günther, M. Haller, and K. G. Paterson, "A formal treatment of end-to-end encrypted cloud storage," in *Proc. CRYPTO*, 2024, pp. 40–74.

[9] J. Hofmann and K. T. Truong, "End-to-end encrypted cloud storage in the wild: A broken ecosystem," in *Proc. ASIACRYPT*, Dec. 2024, pp. 3988–4001.

[10] M. Bellare and C. Namprempre, "Authenticated encryption: Relations among notions and analysis of the generic composition paradigm," in *Proc. ASIACRYPT*, 2000, pp. 531–545.

[11] L. Blackstone, S. Kamara, and T. Moataz, "Revisiting leakage abuse attacks," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, Feb. 2020, pp. 1–18.

[12] D. Q. Viet, A. Yamamura, and H. Tanaka, "Anonymous password-based authenticated key exchange," in *Proc. INDOCRYPT*, 2005, pp. 244–257.

[13] S. Shin, K. Kobara, and H. Imai, "A secure threshold anonymous password-authenticated key exchange protocol," in *Proc. IWSEC*, 2007, pp. 444–458.

[14] J. Yang and Z. Zhang, "A new anonymous password-based authenticated key exchange protocol," in *Proc. INDOCRYPT*, 2008, pp. 200–212.

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on October 22,2026 at 21:15:32 UTC from IEEE Xplore. Restrictions apply.

[15] S. Shin, K. Kobara, and H. Imai, "Very-efficient anonymous password-authenticated key exchange and its extensions," in *Proc. AAECC*, 2009, pp. 149–158.

[16] M. I. G. Vasco, A. L. P. d. Pozo, and C. Soriente, "A key for John doe: Modeling and designing anonymous password-authenticated key exchange protocols," *IEEE Trans. Dependable Secure Comput.*, vol. 18, no. 3, pp. 1336–1353, May 2021.

[17] Y. Yang, J. Zhou, J. Weng, and F. Bao, "A new approach for anonymous password authentication," in *Proc. Annu. Comput. Secur. Appl. Conf.*, Dec. 2009, pp. 199–208.

[18] Y. Yang, J. Zhou, J. W. Wong, and F. Bao, "Towards practical anonymous password authentication," in *Proc. 26th Annu. Comput. Secur. Appl. Conf.*, Dec. 2010, pp. 59–68.

[19] Z. Zhang, K. Yang, X. Hu, and Y. Wang, "Practical anonymous password authentication and TLS with anonymous client authentication," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2016, pp. 1179–1191.

[20] M. Abdalla and D. Pointcheval, "Interactive Diffie–Hellman assumptions with applications to password-based authentication," in *Proc. FC*, 2005, pp. 341–356.

[21] U. Fiege, A. Fiat, and A. Shamir, "Zero knowledge proofs of identity," in *Proc. 19th Annu. ACM Conf. Theory Comput. (STOC)*, New York, NY, USA, 1987, pp. 210–217.

[22] C. Gentry, "Fully homomorphic encryption using ideal lattices," in *Proc. 41st Annu. ACM Symp. Theory Comput. (STOC)*, 2009, pp. 169–178.

[23] D. Freeman, S. Jain, M. Duermuth, B. Biggio, and G. Giacinto, "Who are you? A statistical approach to measuring user authenticity," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2016, pp. 1–15.

[24] A. Guan and C.-M. Chen, "A novel verification scheme to resist online password guessing attacks," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 6, pp. 4285–4293, Nov. 2022.

[25] GitLab. *GitLab.Com Database Incident*. Accessed: Sep. 24, 2026. [Online]. Available: https://about.gitlab.com/blog/gitlab-dot-com-database-incident/

[26] Destiny Dev Team. *Destiny 2 Outage and Rollback*. Accessed: Sep. 24, 2026. [Online]. Available: https://www.bungie.net/7/en/News/article/48723

[27] GitHub. *GitHub Status History*. Accessed: Sep. 24, 2026. [Online]. Available: https://www.githubstatus.com/history?page=3

[28] Reddit User. *Don't Trust iCloud With Your Data–Lost Many Files*. Accessed: Sep. 24, 2026. [Online]. Available: https://www.reddit.com/r/apple/comments/11ex473/donttrusticloudwithyourdatalostmanyfiles/

[29] Reddit User. *Do Not Trust Apple's iCloud or Their Support*. Accessed: Sep. 24, 2026. [Online]. Available: https://www.reddit.com/r/iCloud/comments/1deza5p/donottrustapplesicloudortheirsupport/

[30] Apple Community. *If I downgrade my iCloud storage, do I loose my iCloud data ?* Accessed: Sep. 24, 2026. [Online]. Available: https://discussions.apple.com/thread/255340600?sortBy=rank

[31] M. J. Freedman, Y. Ishai, B. Pinkas, and O. Reingold, "Keyword search and oblivious pseudorandom functions," in *Proc. Theory Cryptogr. Conf.*, 2005, pp. 303–324.

[32] M. Sabt, M. Achemlal, and A. Bouabdallah, "Trusted execution environment: What it is, and what it is not," in *Proc. IEEE Trustcom/BigDataSE/ISPA*, vol. 1, Aug. 2015, pp. 57–64.

[33] D. Catalano and D. Fiore, "Vector commitments and their applications," in *Proc. PKC*, Nara, Japan, 2013, pp. 55–72.

[34] Q. Wang, S. Chen, and Y. Xiang, "Anonymous blockchain-based system for consortium," *ACM Trans. Manag. Inf. Syst.*, vol. 12, no. 3, pp. 1–25, 2021.

[35] W.-G. Tzeng, "Efficient 1-out-n oblivious transfer schemes," in *Proc. PKC*, 2002, pp. 159–171.

[36] D. Boneh and M. Franklin, "Identity-based encryption from the Weil pairing," in *Proc. CRYPTO*, 2001, pp. 213–229.

[37] J. Camenisch and A. Lysyanskaya, "An efficient system for non-transferable anonymous credentials with optional anonymity revocation," in *Proc. EUROCRYPT*, 2001, pp. 93–118.

[38] M. Chase, S. Meiklejohn, and G. Zaverucha, "Algebraic MACs and keyed-verification anonymous credentials," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2014, pp. 1205–1216.

[39] C. Ganesh, C. Orlandi, and D. Tschudi, "Proof-of-stake protocols for privacy-aware blockchains," in *Proc. EUROCRYPT*, 2019, pp. 690–719.

[40] S. Yao and D. Zhang, "An anonymous verifiable random function with applications in blockchain," *Wireless Commun. Mobile Comput.*, vol. 2022, no. 1, Jan. 2022, Art. no. 6467866.

[41] S. Yao and D. Zhang, "An anonymous verifiable random function with unbiasability and constant size proof," *J. Inf. Secur. Appl.*, vol. 83, Jun. 2024, Art. no. 103778.

[42] E.-J. Goh, H. Shacham, N. Modadugu, and D. Boneh, "SiRiUS: Securing remote untrusted storage," in *Proc. NDSS*, 2003, pp. 131–145.

[43] A. Bessani, M. Correia, B. Quaresma, F. André, and P. Sousa, "DepSky: Dependable and secure storage in a cloud-of-clouds," *ACM Trans. Storage*, vol. 9, no. 4, pp. 1–33, 2013.

[44] R. A. Popa, E. Stark, S. Valdez, J. Helfer, N. Zeldovich, and H. Balakrishnan, "Building web applications on top of encrypted data using Mylar," in *Proc. NSDI*, 2014, pp. 157–172.

[45] M. Backendal, F. Günther, and K. G. Paterson, "Puncturable key wrapping and its applications," in *Proc. ASIACRYPT*, 2022, pp. 651–681.

[46] W. Chen and R. Ada Popa, "Metal: A metadata-hiding file-sharing system," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2020, pp. 1–18.

[47] W. Chen, T. Hoang, J. Guajardo, and A. A. Yavuz, "Titanium: A metadata-hiding file-sharing system with malicious security," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2022, pp. 1–18.

[48] P. Das, J. Hesse, and A. Lehmann, "DPaSE: Distributed password-authenticated symmetric-key encryption, or how to get many keys from one password," in *Proc. ACM Asia Conf. Comput. Commun. Secur.*, May 2022, pp. 682–696.

[49] G. T. Davies et al., "Security analysis of the WhatsApp end-to-end encrypted backup protocol," in *Proc. CRYPTO*, 2023, pp. 330–361.

[50] M. Bellare and L. Shea, "Flexible password-based encryption: Securing cloud storage and provably resisting partitioning-oracle attacks," in *Proc. CT-RSA*, 2023, pp. 594–621.

[51] R. C. Merkle, "A digital signature based on a conventional encryption function," in *Proc. CRYPTO*, Santa Barbara, CA, USA. Cham, Switzerland: Springer, 1987, pp. 369–378.

[52] R. Dingledine, N. Mathewson, and P. Syverson, "Tor: The second-generation onion router," in *Proc. USENIX Secur.*, 2004, pp. 303–320.

[53] A. Shamir, "How to share a secret," *Commun. ACM*, vol. 22, pp. 612–613, Nov. 1979.

[54] R. Gennaro and Y. Lindell, "A framework for password-based authenticated key exchange," in *Proc. EUROCRYPT*, 2003, pp. 524–543.

[55] F. F. Yao and Y. L. Yin, "Design and analysis of password-based key derivation functions," in *Proc. CT-RSA*, 2005, pp. 245–261.

[56] N. Tyagi, M. H. Mughees, T. Ristenpart, and I. Miers, "BurnBox: Self-revocable encryption in a world of compelled access," in *Proc. USENIX Secur.*, 2018, pp. 445–461.

[57] J. Pijnenburg and B. Poettering, "Encrypt-to-self: Securely outsourcing storage," in *Proc. ESORICS*, 2020, pp. 635–654.

[58] L. Chen, Y.-N. Li, Q. Tang, and M. Yung, "End-to-same-end encryption: Modularly augmenting an App with an efficient, portable, and blind cloud storage," in *Proc. USENIX Secur.*, 2022, pp. 2353–2370.

[59] Y.-N. Li, Y. Song, Q. Tang, and M. Yung, "End-to-end encrypted git services," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2025, pp. 468–482.

[60] D. Boneh, K. Lewi, H. Montgomery, and A. Raghunathan, "Key homomorphic PRFs and their applications," in *Proc. CRYPTO*, 2013, pp. 410–428.

[61] T. P. Pedersen, "Non-interactive and information-theoretic secure verifiable secret sharing," in *Proc. CRYPTO*, 1991, pp. 129–140.

[62] Protocol Labs. *Merkle DAG*. Accessed: Sep. 24, 2026. [Online]. Available: https://docs.ipfs.tech/concepts/merkle-dag/

[63] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," 2008. Accessed: Sep. 24, 2026. [Online]. Available: https://bitcoin.org/bitcoin.pdf

[64] B. C. Xing, M. Shanahan, and R. Leslie-Hurd, "Intel software guard extensions (Intel SGX) software support for dynamic memory allocation inside an enclave," in *Proc. Hardw. Architectural Support Secur. Privacy*, Jun. 2016, pp. 1–9.

[65] Intel Community. *Microsoft Azure Enables Confidential Computing With Intel SGX*. Accessed: Sep. 24, 2026. [Online]. Available: https://community.intel.com/t5/Blogs/Products-and-Solutions/Security/Microsoft-Azure-enables-confidential-computing-with-Intel-SGX/post/1335021

[66] Google Cloud. *Confidential VM Overview*. Accessed: Sep. 24, 2026. [Online]. Available: https://docs.cloud.google.com/confidential-computing/confidential-vm/docs/confidential-vm-overview?hl=en

[67] D. Wang and P. Wang, "Two birds with one stone: Two-factor authentication with security beyond conventional bound," *IEEE Trans. Dependable Secure Comput.*, vol. 15, no. 4, pp. 708–722, Jul. 2018.

[68] Z. Liu, C. Jiang, and C. Xu, "A portable blind cloud storage scheme against compromised servers," *J. Syst. Archit.*, vol. 146, Jan. 2024, Art. no. 103037.

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on October 22,2026 at 21:15:32 UTC from IEEE Xplore. Restrictions apply.